

Efficient High-Order Masking of FrodoKEMs CDT-Based Gaussian Sampler

Elie Eid, Aurélien Greuet, Nathan Reboud and Rina Zeitoun

IDEMIA Secure Transactions – Cryptography & Security Labs, Courbevoie, France

elie.eid1@idemia.com
aurelien.greuet@idemia.com
nathan.reboud@idemia.com
rina.zeitoun@idemia.com

Abstract. FrodoKEM is a conservative lattice-based KEM based on the Learning With Errors problem. While it was not selected for NIST standardization, it remains a strong candidate for high-security applications and is recommended by several national agencies, including BSI, ANSSI, and the EUCC. Its reliance on CDT-based Gaussian sampling presents a significant challenge for side-channel secure implementations. While recent work by Gérard and Guereau [GG25] has shown that masking FrodoKEM is feasible, the Gaussian sampler remains a major bottleneck, accounting for between 34% and 65% of the execution time. In this work, we introduce a new high-order masking gadget for CDT sampling, provably secure in the ISW probing model and significantly more efficient than previous approaches. We instantiate and evaluate our design on a real-world setup to assess its side-channel resistance in the context of FrodoKEM, using a complete first-order masked implementation on Cortex-M3, which reflects the most relevant practical threat model. Compared with [GG25] at first order, the cost of the sampler is reduced by at least 82% and the number of random generations by at least 69%. Higher-order security is also fully supported through a generic C implementation, with some selected gadgets hand-optimized in assembly to improve efficiency.

Keywords: PQC, Masking, Side-Channels, Gaussian Sampler, CDT, FrodoKEM

1 Introduction

The rise of quantum computers poses a significant threat to classical asymmetric cryptographic systems such as RSA or ECC. Quantum algorithms like Shors [Sho94, PZ03] can efficiently factor large integers and solve the discrete logarithm problem. Once a large-scale quantum computer is ever built, public-key algorithms which security is based on these problems will be effectively compromised.

That's where Post-Quantum Cryptography (PQC) steps in: it seeks to develop, on classical devices, cryptographic algorithms that remain secure against attacks from both classical and quantum adversaries [Ber09].

Acknowledging the critical need for PQC, the NIST launched a standardization initiative in 2016, inviting the cryptographic community to submit candidate schemes for future standards [NIS16].

FrodoKEM, Gaussian Sampler and CDT. Among the many proposals, FrodoKEM, a lattice-based Key Encapsulation Mechanism (KEM) based on the LWE problem, emerged as a promising candidate [ABD⁺23]. It was one of the Round 3 alternate candidates, but was not selected for standardization. Although it was not standardized by NIST, it

continues to earn recognition from the cryptographic community. The German Federal Office for Information Security BSI has recommended FrodoKEM at security levels 3 and 5 as “cryptographically suitable for long-term confidentiality” [BSI21]. Similarly, the French cybersecurity agency ANSSI has endorsed FrodoKEM at the same security levels, describing it as “a valid and conservative option in high security applications where the resulting performance penalty is not prohibitive” [ANS23]. In April 2023, ISO/IEC JTC 1/SC 27/WG 2 approved the progression of FrodoKEM toward standardization as an official mechanism in the ISO/IEC 18033-2 standard, which defines asymmetric encryption algorithms. More recently, in May 2025, the EU Cybersecurity Certification Scheme (EUCS), the European framework for certifying the security of digital systems, added FrodoKEM to its list of recommended cryptographic algorithms [EUC25].

Security of LWE-based schemes like FrodoKEM strongly relies on the Gaussian sampler.

This core routine, introduced in [GPV08], produces lattice points according to a discrete Gaussian distribution, without revealing any information about the underlying private lattice basis. A common way to implement such a sampler is to rely on a Cumulative Distribution Table (CDT).

In general, a CDT is a lookup table that stores the cumulative probabilities of a discrete distribution over a finite support. In our context, the CDT is precomputed from the discrete Gaussian distribution and used to map uniform random inputs to Gaussian samples. Specifically, taking a uniform random value as input, the corresponding sample is the index of the smallest value in the CDT that is larger than the input.

Side-Channel Attacks against the Gaussian Sampler. PQC algorithms are meant to run on a wide variety of devices, including smartcard or secure elements. Such security components must be protected against Side-Channel Attacks (SCA). These attacks exploit unintended information leakage from a system, such as power consumption, electromagnetic (EM) emissions, or timing variations, to extract sensitive data, including cryptographic keys.

In particular, in the context of lattice-based cryptography, the Gaussian sampler can be targeted. As explained in [RRVV14], information obtained from side-channel attacks like timing or power analysis against the Gaussian sampler can lead to key-recovery attacks. Intuitively, the output of the sampler is an error vector whose purpose is to mask a secret, e.g. private key or shared secret. Hence, if this output is known, the secret can be revealed. Even partial information on the sampler values can lead to successful key recovery, by giving hints to improve underlying lattice reduction techniques.

In practice, such information can be obtained by different types of side-channel analysis, even with a constant-time implementation like the following examples. In [GBHLY16], a flush+reload cache attack on the BLISS sampler allows to recover the private key. Likewise, with a single EM trace analysis, [Pes16] is able to recover the output of a sampler with a shuffling countermeasure, and perform a full key recovery. In [KH18], FrodoKEM key generation sampler values are obtained with a single-trace analysis, leading to a key recovery.

Gaussian sampler leakages are also used to filter Falcon signatures to a set depending on the private key and to recover it in [GMRR22, ZLYW23]. Against HAWK, several ways leading to key recovery, depending on the amount of information gained with side-channels, are presented in [GR24].

The shared secret computed in a KEM decapsulation can also be targeted: in [MKK⁺23], sampled values are recovered with a single-trace attack. Then the shared secret is computed by executing the sequel of the decapsulation operations from the sampled values.

One other notable scenario to consider is when a CCA-secure KEM, such as FrodoKEM, is built from a CPA-secure PKE using the Fujisaki-Okamoto (FO) transformation [FO99].

In this setting, the attacker needs a plaintext checking oracle, which provides binary information about the output of the ciphertext decryption in the FO transformation. Using a chosen ciphertext attack and this oracle, the private key can be recovered [RSRCB20, QCZ⁺21, UXT⁺21, RRD⁺23]. Practically, such an oracle can be obtained with SPA or template attacks on the sampler, whose outputs depend on the plaintext value.

All these attacks highlight the critical need to protect Gaussian sampler implementations against side-channel threats.

Securing the Gaussian Sampler against SCA. Shuffling techniques were proposed to thwart attacks on the Gaussian sampler [Saa18]. However, further analysis has demonstrated that it is not sufficient, as other effective attacks remain possible [Pes16].

In [ZLYW23, GMRR22, GR24], ad-hoc mitigation techniques are proposed to reduce leakages. However, they are not proven side-channel protections.

Secure implementations of the Gaussian Sampler were obtained by replacing each operation with its masked equivalent in the standard CDT-based sampling algorithm [GR20, KLS⁺23, GG25]. Nevertheless, these masking techniques introduce a considerable overhead on computational cost.

To prevent timing attacks, these implementations read the entire CDT at each sampling, in order to achieve a constant-time behavior. A natural optimization would be to use a binary decision tree to speed-up the search. However, in its straightforward form, such an approach would induce data-dependent branches or memory accesses, allowing cache attacks, as noted in [KRR⁺18, KLS⁺23]. Consequently, constant-time implementations typically avoid this naive strategy and resort to full-table scans for security. While this is manageable for a small CDT, it becomes increasingly impractical for larger ones, such as in qTESLA [ABB⁺20]. Even for smaller tables, the overhead remains significant: in [GG25], depending on the masking order, the Gaussian sampler accounts for 34% to 65% of the total runtime in a fully masked FrodoKEM-640 decapsulation.

Contributions. We propose a method to improve the SCA-resistant FrodoKEM Gaussian sampler from [GG25]. First, we propose a binary decision tree based approach to reduce the number of comparisons, that is the most expensive step in the masked CDT sampler. Our solution does not introduce any data-dependent branches or memory accesses. Depending on the FrodoKEM parameter set, the number of comparisons is reduced by 50% to 66%. Then, we improve the masked comparison by using a faster underlying secure addition.

We implement our method on an ARM Cortex-M3 smartcard component. Compared with [GG25] at order 1 on the same component, the cycle count for one Gaussian sampler is reduced by 82% to 85% and the number of random generations by 69% to 76%, depending on the FrodoKEM parameter set. To further validate the side-channel resistance, we perform a Test Vector Leakage Assessment (TVLA) on the FrodoKEM-1344 sampler.

In addition, we prove that the masked comparison and the full CDT sampler are t -SNI secure at any order. While gadgets achieving the same functional behavior have already been proposed, to the best of our knowledge, ours are the firsts for which a proof of t -SNI security is given.

Paper organization. The context and general definitions are introduced in Sect. 2. Then Sect. 3 is devoted to our high-order masked FrodoKEM Gaussian sampler: focusing on FrodoKEM-1344, we show how to reduce the number of comparisons in Sect. 3.1, and we describe the masked implementation in Sect. 3.2, including a fast and secure comparison gadget. We give benchmarks of our implementation and comparison with [GG25] in Sect. 4, and results of our TVLA are given in Sect. 5. Finally, Appendix A contains the description of the method for FrodoKEM-640 and FrodoKEM-976.

2 Context

In this section, we provide a brief overview of FrodoKEM to contextualize our work. Rather than detailing the full Key Encapsulation Mechanism, we focus on the discrete CDT-based Gaussian sampler, which plays a central role in the scheme. This sampler is used extensively and has a direct impact on both performance and security. Since our goal is to protect this component against side-channel attacks, we also introduce the concept of masking, along with the t -NI and t -SNI notions from [BBD⁺15b], which are standard tools for analyzing the security of masked implementations in the probing model.

2.1 FrodoKEM algorithm overview

FrodoKEM originates from the Frodo algorithm introduced in [BCD⁺16], and its security is grounded in the difficulty of the Learning With Errors (LWE) problem. Originally introduced by Regev in [Reg05], the LWE problem involves recovering a secret vector $\mathbf{s} \in \mathbb{Z}_q^m$ given a set of samples of the form $(\mathbf{a}, \mathbf{a} \cdot \mathbf{s} + \mathbf{e})$, where $\mathbf{a}$ is a vector uniformly sampled from $\mathbb{Z}_q^m$, $\mathbf{s}$ follows a distribution ϕ and $\mathbf{e}$ is an error term drawn from a noise distribution χ over $\mathbb{Z}_q$. The LWE problem is known to be as hard as worst-case lattice problems, making it a strong foundation for post-quantum cryptographic schemes.

FrodoKEM follows the standard KEM structure, involving key generation, encapsulation, and decapsulation. It achieves IND-CCA security via the (FO) FujisakiOkamoto transformation [FO99], applied to a base Public Key Encryption scheme. This transformation introduces a re-encryption and rejection mechanism during decapsulation, which ensures that an attacker cannot manipulate ciphertexts or recover the shared secret. FrodoKEM is specified with three parameter sets: FrodoKEM-640, FrodoKEM-976, and FrodoKEM-1344, corresponding to NISTs Level 1, 3, and 5 security levels, respectively.

Matrix operations and Gaussian sampling. FrodoKEM uses unstructured matrices with coefficients in $\mathbb{Z}_q$, the ring of integers modulo q , as its core element, meaning that its security does not rely on any additional algebraic structures beyond that of general lattices. The algorithm manipulates matrices of various sizes that depend on two values n and $\bar{n}$, which are chosen according to the desired security level (with larger values offering stronger security), and $\bar{n}$ is always equal to 8. More precisely, the dimensions of these matrices can be one of $\{\bar{n} \times n, n \times \bar{n}, n \times n, \bar{n} \times \bar{n}\}$, depending on the specific step of the algorithm. The core operations consist of matrix generations, modular additions and multiplications in $\mathbb{Z}_q$.

A key part of FrodoKEMs computation involves sampling matrices with discrete Gaussian entries. These matrices are used for secret and error terms during key generation, encapsulation, and decapsulation. In practice, this is done by using a Cumulative Distribution table T_χ to enable efficient sampling. FrodoKEM defines a distinct CDT for each security level; the size s of the table depends on the distribution's standard deviation, which is fixed per parameter set. Each table consists of precomputed cumulative probabilities of a centered discrete Gaussian distribution. These probabilities are scaled to the integer range $[0, 2^{16} - 1]$, so the sampling is performed by comparing a uniform random 16-bit input against the table entries to produce the corresponding Gaussian output, which is an integer in the range $[-(s - 1), s - 1]$, through a table lookup. Table 1 recalls the values of q , n , $\bar{n}$ and the size of the Cumulative Distribution Table of the Gaussian Sampler s for each security level of FrodoKEM. Algorithm 1 describes the procedure for generating a single matrix coefficient.

In FrodoKEM, Alg. 1 is invoked for each individual coefficient of the matrix being sampled. The required uniform randomness is derived from a SHAKE function (SHAKE-128 for FrodoKEM-640 and SHAKE-256 for FrodoKEM-976 and FrodoKEM-1344). The specific CDTs used at each FrodoKEM security level are provided in [Fro23, Table A.4].

Table 1: Some parameters for FrodoKEM.

Parameter set	q	n	$\bar{n}$	s (= size of T_χ)
FrodoKEM-640	2^{15}	640	8	13
FrodoKEM-976	2^{16}	976	8	11
FrodoKEM-1344	2^{16}	1344	8	7

Algorithm 1 GaussianSample(x)**Input:** A 16-bit integer x .**Output:** An integer e between $-(s-1)$ and $(s-1)$.

```

1:  $a \leftarrow x \gg 1$   $\triangleright t$  is a 15-bit integer
2:  $e \leftarrow 0$ 
3: for  $i = 0$  to  $s - 1$  do
4:   (In constant time) If  $a > T_\chi(i)$  then  $e \leftarrow e + 1$ 
5: end for
6:  $e \leftarrow (-1)^{(x \wedge 1)} \cdot e$ 
7: return  $e$ 

```

Table 2 gives the number of calls to Alg. 1 during key generation, encapsulation, and decapsulation for each security level. The table reflects the sampling workload involved at each stage of the protocol.

Table 2: Number of calls to Alg. 1 for FrodoKEM key generation, encapsulation and decapsulation.

Parameter set	Key generation	Encapsulation/Decapsulation
FrodoKEM-640	10 240	10 304
FrodoKEM-976	15 616	15 680
FrodoKEM-1344	21 504	21 568

2.2 Masking and security definitions

Masking is a widely used countermeasure against side-channel attacks that aims to protect sensitive data by splitting it into d shares such that $d - 1$ of them are uniformly and independently distributed. This ensures that any group of $k \leq d - 1$ shares leaks no useful information about the underlying secret, making it difficult for an attacker to recover it by observing intermediate computations.

Depending on the nature of the computation, implementations typically rely on Boolean masking ($x = x_1 \oplus x_2 \oplus \dots \oplus x_d$) or arithmetic masking ($x = x_1 + x_2 + \dots + x_d \bmod q$). In this work, we restrict ourselves to Boolean masking, which is more appropriate for sampling using a CDT.

To formally analyze the security of masked implementations, Ishai, Sahai and Wagner introduced the probing model [ISW03], a theoretical framework that evaluates an adversary's ability to extract secret information by observing a limited number of intermediate variables (probes). In the t -probing model, an attacker can probe up to t intermediate variables of a masked implementation without noise. A circuit is said to be secure at order t if the information gained from any t probes does not reveal additional knowledge about the secret beyond what is already accessible from an equivalent number of input shares.

The probing model has become an essential framework for evaluating the security of masking schemes, enabling rigorous security proofs. A significant advancement in this area is the introduction of Non-Interference (NI) and Strong Non-Interference (SNI) properties, which provide formal criteria for analyzing the security and composability of masked circuits [BBD⁺16]. Below, we recall their definitions.

Definition 1 (*t*-NI security). Let G be a gadget taking d input shares $(x_i)_{1 \leq i \leq d}$ and outputting d shares $(y_i)_{1 \leq i \leq d}$. The gadget G is said *t*-NI secure if for any set $\mathcal{I}$ of t_1 intermediate variables, and any subset $\mathcal{O} \subset [1, d]$ of output indices such that $t_1 + |\mathcal{O}| \leq t$, there exists a subset of input indices $\mathcal{S} \subset [1, d]$ with $|\mathcal{S}| \leq t_1 + |\mathcal{O}|$ such that the t_1 intermediate variables as well as the outputs $y_{|\mathcal{O}|}$ can be perfectly simulated from $x_{|\mathcal{S}|}$; or equivalently, we may say that the gadget G is $(\mathcal{S}, (\mathcal{I}, \mathcal{O}))$ -NI.

Definition 2 (*t*-SNI security). Let G be a gadget taking d input shares $(x_i)_{1 \leq i \leq d}$ and outputting d shares $(y_i)_{1 \leq i \leq d}$. The gadget G is said *t*-SNI secure if for any set $\mathcal{I}$ of t_1 intermediate variables, and any subset $\mathcal{O} \subset [1, d]$ of output indices such that $t_1 + |\mathcal{O}| \leq t$, there exists a subset of input indices $\mathcal{S} \subset [1, d]$ with $|\mathcal{S}| \leq t_1$ such that the t_1 intermediate variables as well as the outputs $y_{|\mathcal{O}|}$ can be perfectly simulated from $x_{|\mathcal{S}|}$; or equivalently, we may write that the gadget G is $(\mathcal{S}, \mathcal{I})$ -SNI.

It is important to note that many masked cryptographic implementations adopt a hybrid approach: designing gadgets that satisfy *t*-NI property and then applying mask refreshing techniques to elevate them to *t*-SNI. This strategy balances efficiency and security, as full *t*-SNI gadgets can introduce significant computation overhead. However, for the sake of clarity, we chose to prove that our sampling gadget is *t*-SNI to ensure that by the composition results from [BBD⁺16], the full algorithm is secure.

3 High-Order Masking of FrodoKEM's Gaussian Sampler

As discussed in Sect. 1, the CDT-based Gaussian sampler of FrodoKEM is vulnerable against Side-Channel Attacks when implemented without proper protections. In this section, we present a masking strategy for FrodoKEMs sampler that achieves arbitrary-order security in the ISW probing model with significantly lower overhead than existing approaches. For the sake of simplicity, we focus on the FrodoKEM-1344 parameter set. This choice is motivated by its alignment with the highest security level among FrodoKEM variants, as well as its relatively small CDT size, which simplifies both the design and the analysis of our techniques. Our constructions for FrodoKEM-640 and FrodoKEM-976 are detailed in Appendix A.

Our masking strategy consists of two main steps. The first step is to reduce the number of comparisons of the form $(a > T_\chi(i))$ in Alg. 1. Indeed, it is a core operation in CDT sampling, that becomes costly when implemented as a masked gadget. We address this challenge by employing a binary decision tree approach that minimizes the number of comparisons without introducing data-dependent branches or memory accesses (see Sect. 3.1). The second step is to replace every operation in the resulting algorithm with its masked equivalent, known as a gadget. To this end, we rely on standard gadgets which we review in Sect. 3.2.1, but we also introduce a new gadget for secure comparison in Sect. 3.2.3. While the secure comparison in [GG25] uses the secure addition from [CGV14], ours is built on a revisited version of the Kogge-Stone adder described in Sect. 3.2.2 to improve performance. We give a security proof of this comparison gadget, which forms the basis for the security proof of the entire masked sampler given in Sect. 3.2.4.

3.1 Reducing the number of comparisons

Recall that in CDT-based sampling, the output integer e represents the number of cumulative probabilities that are smaller than a given uniform random a . That is, e is effectively a counter that is incremented by 1 each time the condition $(a > T_\chi(i))$ holds for a CDT entry $T_\chi(i)$. In our setting, a is obtained from the input uniform random integer x by discarding the least significant bit, since this bit will later determine the sign of the output. A standard constant-time implementation computes this counter iteratively: it goes

through all the table entries, performing one comparison and one conditional increment per element. In an unprotected setting, this sequential process is fast and straightforward. However, in a masked implementation, each comparison must be performed securely using a dedicated gadget, which makes it by far the most expensive operation in the sampler. Indeed, in the masked implementation given in [GG25], the cost of sampling is dominated by the inner loop of secure comparisons. In addition, according to the benchmarks in [GG25], the total time spent in the sampling of the 2 largest matrices is between 34% and 65% of the fully masked FrodoKEM-640 decapsulation. Hence, reducing the number of comparisons directly translates into major performance gains.

Usage of a Binary Decision Tree. The key idea is that the CDT entries are sorted in ascending order, and therefore each comparison output naturally provides a binary decision that splits the remaining possible values of e into two disjoint subsets. This observation allows the linear iteration to be replaced by a binary decision tree, in which each edge corresponds to the outcome of a comparison and each node corresponds to the increment that should be applied to the counter if that branch is taken (for instance, see Fig. 1). Here, "branch is taken" specifically means that the comparison outcome is 1, *i.e.*, the uniform random value a is greater than the threshold at that node. Using such a tree allows the sampler to determine the output value with only $\lceil \log_2(n) \rceil$ comparisons instead of up to $n - 1$ sequential comparisons, where n is the CDT size.

Balanced Tree Structure. To achieve a constant-time implementation, the decision tree should be designed to be perfectly balanced. For this reason, we slightly modify the CDT so that its length is of the form $2^k - 1$. If the original CDT does not meet this condition, for instance in FrodoKEM-640 and FrodoKEM-976, we append a few unreachable threshold values (larger than any valid CDT entry) to complete the table. These dummy entries are never selected but ensure that the tree is structurally regular and that every traversal performs the same number of comparisons.

Decision tree traversal The traversal of the tree begins at the median threshold, corresponding to the midpoint of the CDT, eventually completed to get a $2^k - 1$ length. At each level, the comparison result determines whether the traversal proceeds to the left or right sub-tree: if a is greater than the midpoint threshold, then we move to the right sub-tree and add a power-of-two increment to e , its value depending on the level (starting from 2^{k-1} at the root down to 2^0 at the lowest level). Otherwise, we move to the left sub-tree and add zero (for instance, see Fig. 1). This process continues recursively, halving the search interval at each level until all levels have been traversed.

To implement this in constant time without conditional branches we select the correct threshold of the next tree level using a simple selection formula. For the first level, we compare a directly with the median. For subsequent levels, we apply recursively the formula $a^{(i)} = (b \cdot (v_0 \oplus v_1) \oplus v_0)$, where b is the comparison result from the previous level and v_0, v_1 are the two candidate thresholds for the current level. This ensures that the correct left or right threshold is selected at every level without any branches. Because the increments are distinct powers of two, the updates of e are implemented as simple bitwise Xors, which are both efficient and compatible with boolean masking.

Application to FrodoKEM-1344. Recall that FrodoKEM-1344 uses Alg. 1 with the following CDT from [Fro23, Table A.3]: $T_\chi = (9142, 23462, 30338, 32361, 32725, 32765, 32767)$. The corresponding binary decision tree, shown in Fig. 1, requires only three comparisons instead of six in the sequential version. The last value 32767 is unreachable but ensures a balanced structure with constant comparison depth. In this tree, each node represents the increment applied to the counter e , which is the output of the sampler before applying the

sign. The process starts from $e = 0$ and, depending on the outcome of each comparison, we Xor e with the appropriate power of two. The traversal continues until a leaf node is reached, at which point e represents the final absolute value of the sample. For example, given input $x = 65133$, we obtain $a = x \gg 1 = 32566$, and by traversing the tree we compute $e = 0 \oplus 4 \oplus 0 \oplus 0 = 4$. Since $x \bmod 2 = 1$, the final output is -4 .

This leads to Alg. 2, which operates in constant time. In addition, at each tree level, all nodes are considered together to compute the $a^{(i)}$ s. This avoids the use of branches or data-dependent memory accesses, that may lead to cache attacks.

Comparison with [GG25]. Thanks to this technique, the number of secure comparisons in the Gaussian sampler, compared with [GG25], is

- divided by 3 for FrodoKEM-640 (Alg. 6 in Appendix A),
- divided by 2.5 for FrodoKEM-976 (Alg. 6 in Appendix A),
- divided by 2 for FrodoKEM-1344 (Alg. 2).

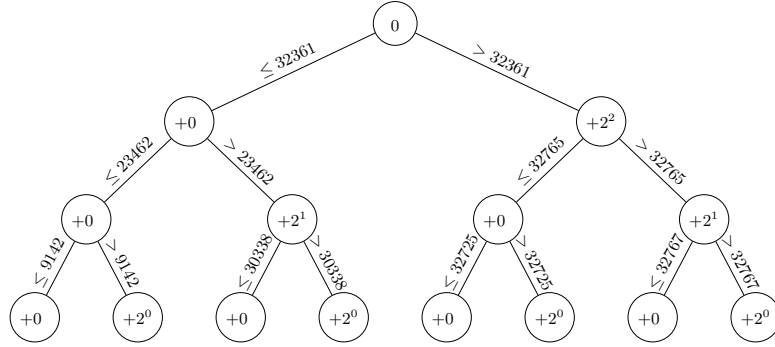


Figure 1: Binary decision tree associated with the Gaussian Sampler of FrodoKEM-1344

Algorithm 2 GaussianSample1344(x). Gaussian Sampler for FrodoKEM-1344

Input: A 16-bit integer x .

Output: GaussianSample(x)

- 1: $a^{(0)} \leftarrow x \gg 1$
 - 2: $b^{(0)} \leftarrow (a^{(0)} > 32361)$ $\triangleright b^{(0)} = 1$ if $a^{(0)} > 32361$, 0 otherwise
 - 3: $a^{(1)} \leftarrow (9307 \cdot b^{(0)} \oplus 23462)$ $\triangleright a^{(1)} = 23462$ or 32765 according to b_0
 - 4: $b^{(1)} \leftarrow (a^{(0)} > a^{(1)})$
 - 5: $a^{(2)} \leftarrow (b^{(1)} \cdot ((b^{(0)} \cdot 21790) \oplus 21812)) \oplus (b^{(0)} \cdot 23651) \oplus 9142$
 - 6: $b^{(2)} \leftarrow (a^{(0)} > a^{(2)})$
 - 7: **return** $(-1)^{x \wedge 1} \cdot ((b^{(0)} \ll 2) \oplus (b^{(1)} \ll 1) \oplus b^{(2)})$
-

3.2 Improving secure comparisons and CDT Sampler Masking

To ensure t -order security in the probing model, we represent each sensitive variable using $d = t + 1$ shares, where $d - 1$ of these shares are uniform and independently distributed. We then replace every subroutine in Alg. 2 with a corresponding masked gadget, which securely processes the given shares to get the same functional behavior, the output being given by d shares as well. We first describe gadgets already available in the literature. Then we present and prove the security of a comparison gadget based on a secure addition

built from Kogge-Stone adder from [BBE⁺18]. Compared with [GG25], where the secure addition is based on [CGV14], the number of masked operation is reduced. Finally, we combine these gadgets to achieve the security of our Gaussian sampler.

3.2.1 Standard gadgets

Our new gadget and the sampler rely on the following gadgets:

- **secAnd**: t -SNI bitwise **and** on Boolean sharings (**SecMult** in [BBD⁺16]),
- **refresh**: t -SNI Boolean sharing refreshing (**RefreshM** in [BBD⁺16]),
- affine gadgets, such as **secXor** and **secShift**, which apply the affine operation share by share and are t -NI (see [BBD⁺15b, Theorem 1]).

For simplicity, we use the notation $(x_i \ll s)_{1 \leq i \leq d}$ (resp. $(x_i \gg s)_{1 \leq i \leq d}$) to denote a secure left (resp. right) shift by s positions on the masked value $(x_i)_{1 \leq i \leq d}$. Moreover, xoring a masked value $(x)_{1 \leq i \leq d}$ with a constant c is achieved by xoring one share with c . Similarly, a bitwise **and** between a masked value $(x)_{1 \leq i \leq d}$ and a constant c is computed as $(x \wedge c)_{1 \leq i \leq d}$.

3.2.2 KoggeStoneCarry

The gadget **KoggeStoneCarry** will be used to build the secure comparison gadget in Sect 3.2.3. It is based on the Kogge-Stone adder built in [BBE⁺18] and previously proven to satisfy the t -NI property in the probing model. We adapt this gadget by removing its final computation step, namely $x \oplus y \oplus (g \ll 1)$, since in the context of secure comparison, only the propagation of the carry bits represented by the value g is required. Specifically, when **KoggeStoneCarry** is applied to inputs x and $(-y - 1)$, the most significant bit of the output g indicates whether $x > y$, as the comparison is reduced to checking the sign of $x - (y + 1)$. We prove that this adapted version of the adder satisfies the stronger t -SNI property. Our gadget, **KoggeStoneCarry**, is detailed in Alg. 3.

For the sake of clarity, the security proof of Alg. 3 is split in 3 steps. We first prove the security of one iteration of the inner loop in Lemma 1. Then, Lemma 2 gives a security proof of all loop iterations. Finally, the security proof for Alg. 3 is given by Lemma 3.

Algorithm 3 **KoggeStoneCarry** $((x_i)_{1 \leq i \leq d}, (y_i)_{1 \leq i \leq d}, w)$

Input: Boolean shares $(x_i)_{1 \leq i \leq d}$ and $(y_i)_{1 \leq i \leq d}$ of w -bit integers x and y .

Output: Boolean shares $(g_i)_{1 \leq i \leq d}$ of g such that $(x + y) \bmod 2^w = x \oplus y \oplus (g \ll 1)$.

```

1:  $(p_i)_{1 \leq i \leq d} \leftarrow \text{secXor}((x_i)_{1 \leq i \leq d}, (y_i)_{1 \leq i \leq d})$ 
2:  $(g_i)_{1 \leq i \leq d} \leftarrow \text{secAnd}((x_i)_{1 \leq i \leq d}, (y_i)_{1 \leq i \leq d}, w)$ 
3: for  $j = 1$  to  $\lceil \log_2(w - 1) \rceil - 1$  do
4:    $(a_i)_{1 \leq i \leq d} \leftarrow (g_i \ll 2^{j-1})_{1 \leq i \leq d}$ 
5:    $(a_i)_{1 \leq i \leq d} \leftarrow \text{secAnd}((a_i)_{1 \leq i \leq d}, (p_i)_{1 \leq i \leq d}, w)$ 
6:    $(g_i)_{1 \leq i \leq d} \leftarrow \text{secXor}((g_i)_{1 \leq i \leq d}, (a_i)_{1 \leq i \leq d})$ 
7:    $(a'_i)_{1 \leq i \leq d} \leftarrow (p_i \ll 2^{j-1})$ 
8:    $(a'_i)_{1 \leq i \leq d} \leftarrow \text{refresh}((a'_i)_{1 \leq i \leq d}, w)$ 
9:    $(p_i)_{1 \leq i \leq d} \leftarrow \text{secAnd}((p_i)_{1 \leq i \leq d}, (a'_i)_{1 \leq i \leq d}, w)$ 
10: end for
11:  $(a_i)_{1 \leq i \leq d} \leftarrow (g_i \ll 2^{\lceil \log_2(w-1) \rceil - 1})_{1 \leq i \leq d}$ 
12:  $(a_i)_{1 \leq i \leq d} \leftarrow \text{secAnd}((a_i)_{1 \leq i \leq d}, (p_i)_{1 \leq i \leq d}, w)$ 
13:  $(g_i)_{1 \leq i \leq d} \leftarrow \text{secXor}((g_i)_{1 \leq i \leq d}, (a_i)_{1 \leq i \leq d})$ 
14: return  $(g_i)_{1 \leq i \leq d}$ 

```

Lemma 1 (Security of one inner loop iteration). *Let G be the gadget corresponding to an iteration of Alg. 3, and illustrated in Fig. 2. Let $(g_i)_{1 \leq i \leq d}$ and $(p_i)_{1 \leq i \leq d}$ be input shares of G and $(g'_i)_{1 \leq i \leq d}$ and $(p'_i)_{1 \leq i \leq d}$ be output shares. For any set of t_1 intermediate variables and any subsets $\mathcal{O}_1$ and $\mathcal{O}_2$ of output indices such that $t_1 + |\mathcal{O}_1| < d$ and $t_1 + |\mathcal{O}_2| < d$, there exist two subsets of input indices $\mathcal{S}$ and $\mathcal{S}'$ with $|\mathcal{S}| \leq t_1 + |\mathcal{O}_1|$ and $|\mathcal{S}'| \leq t_1$ such that the t_1 intermediate variables as well as the outputs $g'_{|\mathcal{O}_1|}$ and $p'_{|\mathcal{O}_2|}$ can be perfectly simulated from $g_{|\mathcal{S}|}$ and $p_{|\mathcal{S}'|}$.*

Proof. The proof follows the process described in [BBD⁺15a, Sect. 4.1]: we prove that the composition of all subgadgets of Fig. 2 gives the desired result.

We label the subgadgets from 1 to 6 as shown in Fig. 2. Let $\mathcal{I}$ be a set of indices corresponding to observations of intermediate variables such that $|\mathcal{I}| \leq t_1$ and $\mathcal{O}_1$ and $\mathcal{O}_2$ be two sets of output indices such that $|\mathcal{I}| + |\mathcal{O}_1| < d$ and $|\mathcal{I}| + |\mathcal{O}_2| < d$. Consider a partition $\mathcal{I} = \mathcal{I}^1 \cup \dots \cup \mathcal{I}^6$, depending on the subgadget in which the observation occurs.

Gadget 1 As $|\mathcal{I}^1| + |\mathcal{O}_1| < d$ and since gadget 1 is affine, there exist two sets of indices $\mathcal{S}_1^1$ and $\mathcal{S}_2^1$ with $|\mathcal{S}_1^1|, |\mathcal{S}_2^1| \leq |\mathcal{I}^1| + |\mathcal{O}_1|$, such that the gadget is $((\mathcal{S}_1^1, \mathcal{S}_2^1), (\mathcal{I}^1, \mathcal{O}_1))$ -NI.

Gadget 2 From the previous step, $|\mathcal{I}^2| + |\mathcal{S}_2^1| \leq |\mathcal{I}^2| + |\mathcal{I}^1| + |\mathcal{O}_1| < d$ and since **secAnd** is t -SNI there exist two sets of indices $\mathcal{S}_1^2$ and $\mathcal{S}_2^2$ with $|\mathcal{S}_1^2|, |\mathcal{S}_2^2| \leq |\mathcal{I}^2|$, such that the gadget is $((\mathcal{S}_1^2, \mathcal{S}_2^2), \mathcal{I}^2)$ -SNI.

Gadget 3 From the previous step, $|\mathcal{I}^3| + |\mathcal{S}_1^2| \leq |\mathcal{I}^3| + |\mathcal{I}^2| < d$ and since gadget 3 is affine, there exists a set of indices $\mathcal{S}^3$ with $|\mathcal{S}^3| \leq |\mathcal{I}^3| + |\mathcal{S}_1^2|$, such that the gadget is $(\mathcal{S}^3, (\mathcal{I}^3, \mathcal{S}_1^2))$ -NI.

Gadget 4 As $|\mathcal{I}^4| + |\mathcal{O}_2| < d$ and since **secAnd** is t -SNI there exist two sets of indices $\mathcal{S}_1^4$ and $\mathcal{S}_2^4$ with $|\mathcal{S}_1^4|, |\mathcal{S}_2^4| \leq |\mathcal{I}^4|$, such that the gadget is $((\mathcal{S}_1^4, \mathcal{S}_2^4), \mathcal{I}^4)$ -SNI.

Gadget 5 From the previous step, $|\mathcal{I}^5| + |\mathcal{S}_1^4| \leq |\mathcal{I}^5| + |\mathcal{I}^4| < d$ and since the gadget is t -SNI there exists a set of indices $\mathcal{S}^5$ with $|\mathcal{S}^5| \leq |\mathcal{I}^5|$, such that the gadget is $(\mathcal{S}^5, \mathcal{I}^5)$ -SNI.

Gadget 6 From the previous step, $|\mathcal{I}^6| + |\mathcal{S}^5| \leq |\mathcal{I}^6| + |\mathcal{I}^5| < d$ and since gadget 6 is affine, there exists a set of indices $\mathcal{S}^6$ with $|\mathcal{S}^6| \leq |\mathcal{I}^6| + |\mathcal{S}^5|$, such that the gadget is $(\mathcal{S}^6, (\mathcal{I}^6, \mathcal{S}^5))$ -NI.

Each of the preceding steps guarantees the existence of a simulator for every gadget. Let $\mathcal{S} = \mathcal{S}_1^1 \cup \mathcal{S}^3$ and $\mathcal{S}' = \mathcal{S}_2^2 \cup \mathcal{S}^6 \cup \mathcal{S}_2^4$. We then compose all the simulators to perfectly simulate the gadget from input shares corresponding to indices in $\mathcal{S}$ and $\mathcal{S}'$. Furthermore, we have $|\mathcal{S}| \leq |\mathcal{S}_1^1| + |\mathcal{S}^3| \leq |\mathcal{I}^1| + |\mathcal{O}_1| + |\mathcal{I}^3| + |\mathcal{I}^2| \leq t_1 + |\mathcal{O}_1|$ and $|\mathcal{S}'| \leq |\mathcal{S}_2^2| + |\mathcal{S}^6| + |\mathcal{S}_2^4| \leq |\mathcal{I}^2| + |\mathcal{I}^5| + |\mathcal{I}^6| + |\mathcal{I}^4| \leq t_1$, which concludes the proof. $\square$

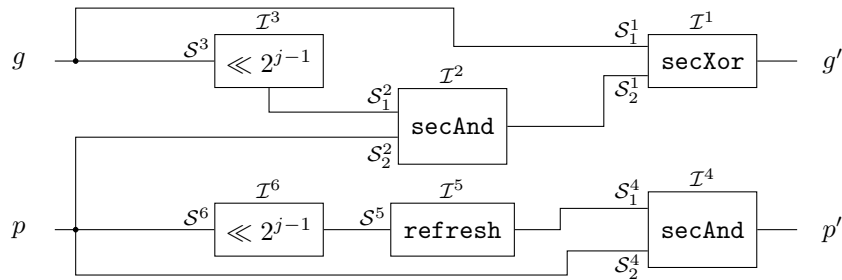


Figure 2: Kogge-Stone one inner loop iteration gadget

Lemma 2 (Security of full inner loop). *Let G be the gadget corresponding to steps 3-10 of Alg. 3, and illustrated in Fig. 3. Let $(g_i)_{1 \leq i \leq d}$ and $(p_i)_{1 \leq i \leq d}$ be input shares of G and $(g'_i)_{1 \leq i \leq d}$ and $(p'_i)_{1 \leq i \leq d}$ be output shares. For any set of t_1 intermediate variables and any subsets $\mathcal{O}_1$ and $\mathcal{O}_2$ of output indices such that $t_1 + |\mathcal{O}_1| < d$ and $t_1 + |\mathcal{O}_2| < d$, there exist two subsets of input indices $\mathcal{S}$ and $\mathcal{S}'$ with $|\mathcal{S}| \leq t_1 + |\mathcal{O}_1|$ and $|\mathcal{S}'| \leq t_1$ such that the*

t_1 intermediate variables as well as the outputs $g'_{|\mathcal{O}_1}$ and $p'_{|\mathcal{O}_2}$ can be perfectly simulated from $g_{|\mathcal{S}}$ and $p_{|\mathcal{S}'}$.

Proof. We use Lemma 1 to show that the composition of all subgadgets of Fig. 3 gives the desired result. For clarity, we label the subgadgets from 1 to W as shown in Fig. 3. We denote by KSI_i the gadget corresponding to the i -th inner loop iteration in Alg. 3. Let $\mathcal{I}$ be a set of indices corresponding to observations of intermediate variables such that $|\mathcal{I}| \leq t_1$ and $\mathcal{O}_1$ and $\mathcal{O}_2$ be two sets of output indices such that $|\mathcal{I}| + |\mathcal{O}_1| < d$ and $|\mathcal{I}| + |\mathcal{O}_2| < d$. Consider a partition $\mathcal{I} = \mathcal{I}^1 \cup \dots \cup \mathcal{I}^W$, depending on the subgadget in which the observation occurs and write $\mathcal{S}_1^0 = \mathcal{O}_1$ and $\mathcal{S}_2^0 = \mathcal{O}_2$.

For all $i \in [1, W]$, we prove by induction the following statement: for all $i \in [1, W]$, there exist two sets of indices $\mathcal{S}_1^i$ and $\mathcal{S}_2^i$ with $|\mathcal{S}_1^i| \leq |\mathcal{I}^i| + |\mathcal{S}_1^{i-1}|$ and $|\mathcal{S}_2^i| \leq |\mathcal{I}^i|$, such that the gadget KSI_i is perfectly simulated from its input shares corresponding to $\mathcal{S}_1^i$ and $\mathcal{S}_2^i$. For $i = 1$, as $|\mathcal{I}^1| + |\mathcal{S}_1^0| \leq |\mathcal{I}| + |\mathcal{O}_1| < d$ and $|\mathcal{I}^1| + |\mathcal{S}_2^0| \leq |\mathcal{I}| + |\mathcal{O}_2| < d$, then we can apply Lemma 1 to obtain the desired result. Now, let $i \in [2, W]$. We assume that the statement is true for all $j \in [1, i-1]$ and let us prove that it is true for i . Since

$$|\mathcal{I}^i| + |\mathcal{S}_1^{i-1}| \leq |\mathcal{I}^i| + |\mathcal{I}^{i-1}| + |\mathcal{S}_1^{i-2}| \leq \dots \leq \sum_{j=1}^i |\mathcal{I}^j| + |\mathcal{S}_1^0| \leq |\mathcal{I}| + |\mathcal{O}_1| < d$$

and $|\mathcal{I}^i| + |\mathcal{S}_2^{i-1}| \leq |\mathcal{I}^i| + |\mathcal{I}^{i-1}| \leq |\mathcal{I}| < d$, then Lemma 1 gives the desired result.

The above statement guarantees the existence of a simulator for each gadget KSI_i . Take $\mathcal{S} = \mathcal{S}_1^W$ and $\mathcal{S}' = \mathcal{S}_2^W$. We then compose all the simulators of KSI_i to perfectly simulate their composition from input shares corresponding to $\mathcal{S}$ and $\mathcal{S}'$. Furthermore, we have

$$|\mathcal{S}| = |\mathcal{S}_1^W| \leq |\mathcal{I}^W| + |\mathcal{S}_1^{W-1}| \leq \dots \leq \sum_{i=1}^W |\mathcal{I}^i| + |\mathcal{O}_1| \leq |\mathcal{I}| + |\mathcal{O}_1| \leq t_1 + |\mathcal{O}_1|$$

and $|\mathcal{S}'| = |\mathcal{S}_2^W| \leq |\mathcal{I}^W| \leq |\mathcal{I}| \leq t_1$, which concludes the proof. $\square$

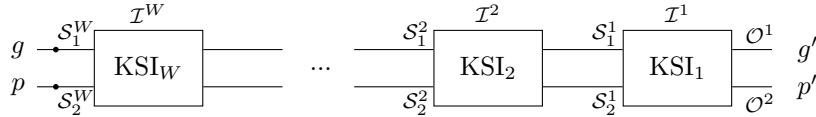


Figure 3: Kogge-Stone Loop gadget

Lemma 3 (*KoggeStoneCarry* (Alg. 3) is t -SNI). *Let $(x_i)_{1 \leq i \leq d}$ and $(y_i)_{1 \leq i \leq d}$ be input shares of the *KoggeStoneCarry* gadget, and let $(g_i)_{1 \leq i \leq d}$ be output shares. For any set t_1 of intermediate variables and any subset $\mathcal{O}$ of output indices such that $t_1 + |\mathcal{O}| < d$, there exist two subsets of input indices $\mathcal{S}$ and $\mathcal{S}'$ with $|\mathcal{S}| \leq t_1$ and $|\mathcal{S}'| \leq t_1$ such that the t_1 intermediate variables as well as the outputs $g_{|\mathcal{O}}$ can be perfectly simulated from $x_{|\mathcal{S}}$ and $y_{|\mathcal{S}'}$.*

Proof. The proof follows the same reasoning as the proof of Lemma 1. We use Lemma 1 and Lemma 2 to show that the composition of all subgadgets of Fig. 4 gives the desired result. Let $\mathcal{I}$ be a set of indices corresponding to observations of intermediate variables such that $|\mathcal{I}| \leq t_1$ and $\mathcal{O}$ be a set of output indices such that $|\mathcal{I}| + |\mathcal{O}| < d$. Consider a partition $\mathcal{I} = \mathcal{I}^1 \cup \dots \cup \mathcal{I}^6$, depending on the gadget in which the observation occurs.

Gadgets 1,2 and 3 The composition of these gadgets is already handled in the proof of Lemma 1 (steps 1, 2 and 3). More precisely, let $\mathcal{I}' = \mathcal{I}^1 \cup \mathcal{I}^2 \cup \mathcal{I}^3$, then there exist two

sets of indices I and J with $|I| \leq |\mathcal{I}'| + |\mathcal{O}|$ and $|J| \leq |\mathcal{I}'|$ such that the composition is $((I, J), (\mathcal{I}', \mathcal{O}))$ -NI.

Gadget 4 From the previous step, $|\mathcal{I}^4| + |I| \leq |\mathcal{I}^4| + |\mathcal{I}'| + |\mathcal{O}| < d$ and $|\mathcal{I}^4| + |J| \leq |\mathcal{I}| < d$, and according to Lemma 2, there exist two sets of indices $\mathcal{S}_1^4$ and $\mathcal{S}_2^4$ with $|\mathcal{S}_1^4| \leq |\mathcal{I}^4| + |I|$ and $|\mathcal{S}_2^4| \leq |\mathcal{I}^4|$ such that the composition is $((\mathcal{S}_1^4, \mathcal{S}_2^4), (\mathcal{I}^4, (I, J)))$ -NI.

Gadget 5 From the previous step, $|\mathcal{I}^5| + |\mathcal{S}_2^4| \leq t_1 < d$ and since gadget 5 is affine, there exist two sets of indices $\mathcal{S}_1^5$ and $\mathcal{S}_2^5$ with $|\mathcal{S}_1^5|, |\mathcal{S}_2^5| \leq |\mathcal{I}^5| + |\mathcal{S}_2^4|$, such that the gadget is $((\mathcal{S}_1^5, \mathcal{S}_2^5), (\mathcal{I}^5, \mathcal{S}_2^4))$ -NI.

Gadget 6 From Step 4, $|\mathcal{I}^6| + |\mathcal{S}_1^4| \leq t_1 + |\mathcal{O}| < d$ and since **secAnd** is t -SNI, there exist two sets of indices $\mathcal{S}_1^6$ and $\mathcal{S}_2^6$ with $|\mathcal{S}_1^6|, |\mathcal{S}_2^6| \leq |\mathcal{I}^6|$, such that the gadget is $((\mathcal{S}_1^6, \mathcal{S}_2^6), \mathcal{I}^6)$ -SNI.

Each of the preceding steps guarantees the existence of a simulator for each gadget. Let $\mathcal{S} = \mathcal{S}_1^5 \cup \mathcal{S}_2^6$ and $\mathcal{S}' = \mathcal{S}_2^5 \cup \mathcal{S}_1^6$. We then compose all the simulators to perfectly simulate the gadget from input shares corresponding to indices in $\mathcal{S}$ and $\mathcal{S}'$. Furthermore, we have $|\mathcal{S}| \leq |\mathcal{S}_1^5| + |\mathcal{S}_2^6| \leq |\mathcal{I}^5| + |\mathcal{I}^4| + |\mathcal{I}^6| \leq t_1$ and $|\mathcal{S}'| \leq |\mathcal{S}_2^5| + |\mathcal{S}_1^6| \leq |\mathcal{I}^5| + |\mathcal{I}^4| + |\mathcal{I}^6| \leq t_1$, which concludes the proof. $\square$

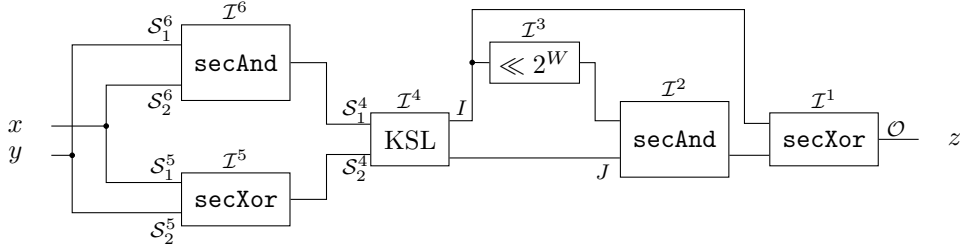


Figure 4: KoggeStoneCarry gadget

3.2.3 Secure Comparison

This section is devoted to the secure comparison between two values, each one given by a Boolean sharing. The construction is detailed in Alg. 4 and its security proof in the probing model is established in Lemma 4, where we prove that the gadget is t -SNI.

The idea behind Alg. 4 is to compare x and y of bitsize w by computing the sign bit of the sum $x + (-y - 1)$. Indeed, this bit is set iff the addition result is negative. From the identity $-y = \text{not}(y) + 1$, that is $-y - 1 = \text{not}(y)$, a sharing of $(-y - 1)$ is obtained by replacing the first share y_1 by $y_1 \oplus (2^{w+1} - 1)$. This transformation both negates y and extends it to $(w + 1)$ bits by setting the upper bit to 1, to ensure correct sign propagation during the addition with x . The sum is then handled with **KoggeStoneCarry**, which computes the carry propagation g of the addition. Since $(g \ll 1) = (x + (-y - 1)) \oplus x \oplus (-y - 1)$, we observe that $x > y$ iff the MSB of g is set to one, this is because the MSB of the result $x + (-y - 1)$ considered as a $w + 1$ -bit integer is zero iff $x > y$, while the MSB of $(-y - 1)$ is always equal to one.

Lemma 4 (**secCompare** (Alg. 4) is t -SNI). *Let $(x_i)_{1 \leq i \leq d}$ and $(y_i)_{1 \leq i \leq d}$ be input shares of the **secCompare** gadget, and let $(b_i)_{1 \leq i \leq d}$ be output shares. For any set t_1 of intermediate variables and any subset $\mathcal{O}$ of output indices such that $t_1 + |\mathcal{O}| < d$, there exist two subsets of input indices $\mathcal{S}$ and $\mathcal{S}'$ with $|\mathcal{S}| \leq t_1$ and $|\mathcal{S}'| \leq t_1$ such that the t_1 intermediate variables as well as the outputs $b_{|\mathcal{O}|}$ can be perfectly simulated from $x_{|\mathcal{S}|}$ and $y_{|\mathcal{S}'|}$.*

Proof. We use Lemma 3 to show that the composition of all gadgets of Fig. 5 gives the desired result. Let $\mathcal{I}$ be a set of indices corresponding to observations of intermediate

Algorithm 4 $\text{secCompare}((x_i)_{1 \leq i \leq d}, (y_i)_{1 \leq i \leq d}, w)$

Input: Boolean shares $(x_i)_{1 \leq i \leq d}$ and $(y_i)_{1 \leq i \leq d}$ of w -bit integers x and y .

Output: Boolean shares $(b_i)_{1 \leq i \leq d}$ of a bit b such that $b = 1$ if $x > y$ and $b = 0$ otherwise.

- 1: $y_1 \leftarrow y_1 \oplus (2^{w+1} - 1)$ $\triangleright$ Complement of y seen as a $(w + 1)$ -bit integer
 - 2: $(g_i)_{1 \leq i \leq d} \leftarrow \text{KoggeStoneCarry}((x_i)_{1 \leq i \leq d}, (y_i)_{1 \leq i \leq d}, w + 1)$.
 - 3: $(g_i)_{1 \leq i \leq d} \leftarrow (g_i \gg (w - 1))_{1 \leq i \leq d}$
 - 4: **return** $(g_i \wedge 1)_{1 \leq i \leq d}$
-

variables such that $|\mathcal{I}| \leq t_1$ and $\mathcal{O}$ be a set of output indices such that $|\mathcal{I}| + |\mathcal{O}| < d$. Consider a partition $\mathcal{I} = \mathcal{I}^1 \cup \mathcal{I}^2 \cup \mathcal{I}^3$, depending on the gadget in which the observation occurs.

Gadget 1 The gadget represents steps 3-4 of Alg. 4 which is t -NI. As $|\mathcal{I}^1| + |\mathcal{O}| \leq |\mathcal{I}| + |\mathcal{O}| < d$, then there exists a set of indices $\mathcal{S}^1$ with $|\mathcal{S}^1| \leq |\mathcal{I}^1| + |\mathcal{O}|$, such that Gadget 1 is $(\mathcal{S}^1, (\mathcal{I}^1, \mathcal{O}))$ -NI.

Gadget 2 From the previous step, $|\mathcal{I}^2| + |\mathcal{S}^1| \leq |\mathcal{I}| + |\mathcal{O}| < d$ and since KoggeStoneCarry is t -SNI by Lemma 3, there exist two sets of indices $\mathcal{S}_1^2$ and $\mathcal{S}_2^2$ with $|\mathcal{S}_1^2|, |\mathcal{S}_2^2| \leq |\mathcal{I}^2|$, such that Gadget 2 is $((\mathcal{S}_1^2, \mathcal{S}_2^2), \mathcal{I}^2)$ -SNI.

Gadget 3 The gadget represents Step 1 of Alg. 4 which is t -NI. From Step 2, $|\mathcal{I}^3| + |\mathcal{S}_2^2| \leq |\mathcal{I}| < d$, then there exists a set of indices $\mathcal{S}^3$ with $|\mathcal{S}^3| \leq |\mathcal{I}^3| + |\mathcal{S}_2^2|$, such that Gadget 3 is $(\mathcal{S}^3, (\mathcal{I}^3, \mathcal{S}_2^2))$ -NI.

Each of the preceding steps guarantees the existence of a simulator for each gadget. Let $\mathcal{S} = \mathcal{S}_1^2$ and $\mathcal{S}' = \mathcal{S}^3$. We then compose all the simulators to perfectly simulate the gadget from input shares corresponding to indices in $\mathcal{S}$ and $\mathcal{S}'$. Furthermore, we have $|\mathcal{S}| \leq |\mathcal{I}^2| \leq t_1$ and $|\mathcal{S}'| \leq |\mathcal{I}^3| + |\mathcal{S}_2^2| \leq |\mathcal{I}^3| + |\mathcal{I}^2| \leq t_1$, which concludes the proof. $\square$

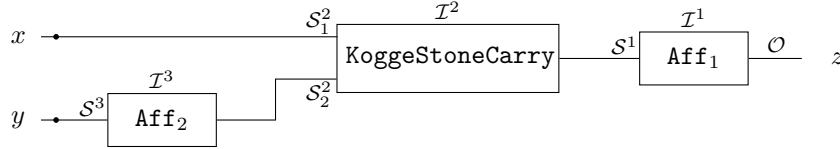


Figure 5: Secure comparison gadget

3.2.4 CDT-based Gaussian sampler

The masked version of Alg. 2 is provided in Alg. 5, with its security formally proven in Theorem 1.

Algorithm 5 is obtained by systematically replacing each operation in the unmasked algorithm with its masked equivalent and inserting mask refreshing where necessary to ensure t -SNI security. Concretely, each comparison is done with the secCompare gadget, each bitwise **ands** between shared variables are handled by secAnd , bitwise **ands** between a shared variable and a constant are computed by applying the **and** with the constant on each share, and bitwise **xors** and logical shifts are done share by share.

The final step of sampling consists in computing the signed output $(-1)^s \cdot r$ with $s = x \wedge 1$ (line 7 in Alg. 2). This determines the sign of the result and corresponds to lines 1114 in Alg. 5. Since $s = 0$ or 1, the two's complement identity $-r = \text{not}(r) + 1$ allows to rewrite $(-1)^s \cdot r$ as $(\text{not}(r) + s) \oplus \text{not}(-s)$. Let $g = \text{KoggeStoneCarry}(\text{not}(r), s, 4)$. Then

$$(\text{not}(r) + s) \oplus \text{not}(-s) = (g \ll 1) \oplus \text{not}(r) \oplus s \oplus \text{not}(-s) = (g \ll 1) \oplus r \oplus s \oplus (-s),$$

given that r is a 3-bit value (so $\text{not}(r)$ is extended to 4 bits to capture the sign bit). Moreover, $s = 0$ or 1 . As a 4-bit value, $s \oplus (-s) = 0$ or $1 \oplus (-1) = 1 \oplus 0xF = 0xE$. In both cases, $s \oplus (-s) = 0xE \cdot s$.

Algorithm 5 `secGaussianSample-1344` $((x_i)_{1 \leq i \leq d})$. Masked Gaussian Sampler for FrodoKEM-1344

Input: Boolean shares $(x_i)_{1 \leq i \leq d}$ of a 16-bit integer x .

Output: Boolean shares $(r_i)_{1 \leq i \leq d}$ of `GaussianSample` (x) over 4 bits.

```

1:  $(a_i^{(0)}) \leftarrow (x_i \gg 1)_{1 \leq i \leq d}$ 
2:  $(s_i)_{1 \leq i \leq d} \leftarrow (x_i \wedge 1)_{1 \leq i \leq d}$ 
3:  $(b_i^{(0)})_{1 \leq i \leq d} \leftarrow \text{secCompare}((a_i^{(0)})_{1 \leq i \leq d}, (32361, 0, \dots, 0))$ 
4:  $(a_i^{(1)})_{1 \leq i \leq d} \leftarrow ((b_i^{(0)} \cdot 9307))_{1 \leq i \leq d} \oplus (23462, 0, \dots, 0)$ 
5:  $(b_i^{(1)})_{1 \leq i \leq d} \leftarrow \text{secCompare}((a_i^{(0)})_{1 \leq i \leq d}, (a_i^{(1)})_{1 \leq i \leq d})$ 
6:  $(a_i^{(2)})_{1 \leq i \leq d} \leftarrow ((b_i^{(0)} \cdot 21790))_{1 \leq i \leq d} \oplus (21812, 0, \dots, 0)$ 
7:  $(a_i^{(2)})_{1 \leq i \leq d} \leftarrow \text{secAnd}((a_i^{(2)})_{1 \leq i \leq d}, (b_i^{(1)} \cdot 0xFFFF)_{1 \leq i \leq d})$ 
8:  $(a_i^{(2)})_{1 \leq i \leq d} \leftarrow \text{secXor}(((b_i^{(0)} \cdot 23651)_{1 \leq i \leq d}, (a_i^{(2)})_{1 \leq i \leq d}) \oplus (9142, 0, \dots, 0))$ 
9:  $(b_i^{(2)})_{1 \leq i \leq d} \leftarrow \text{secCompare}((a_i^{(0)})_{1 \leq i \leq d}, (a_i^{(2)})_{1 \leq i \leq d})$ 
10:  $(r_i)_{1 \leq i \leq d} \leftarrow \text{secXor}(\text{secXor}((b_i^{(0)} \ll 2)_{1 \leq i \leq d}, (b_i^{(1)} \ll 1)_{1 \leq i \leq d}), (b_i^{(2)})_{1 \leq i \leq d})$ 
11:  $(g_i)_{1 \leq i \leq d} \leftarrow \text{KoggeStoneCarry}((r_i)_{1 \leq i \leq d} \oplus (0xF, 0, \dots, 0), (s_i)_{1 \leq i \leq d}, 4)$ 
12:  $(s_i)_{1 \leq i \leq d} \leftarrow \text{refresh}((s_i \cdot 0xE)_{1 \leq i \leq d}, 4)$ 
13:  $(r_i)_{1 \leq i \leq d} \leftarrow \text{secXor}(\text{secXor}((r_i)_{1 \leq i \leq d}, (g_i \ll 1)_{1 \leq i \leq d}), (s_i)_{1 \leq i \leq d})$ 
14: return  $(r_i)_{1 \leq i \leq d}$ 

```

Theorem 1 (t -SNI of `secGaussianSample-1344` (Alg. 5)). *Let $(x_i)_{1 \leq i \leq d}$ be the input shares of the `secGaussianSample-1344` gadget, and let $(r_i)_{1 \leq i \leq d}$ be the output shares. For any set of t_1 intermediate variables and any subset $\mathcal{O}$ of output indices such that $t_1 + |\mathcal{O}| < d$, there exists a subset $\mathcal{S}$ of indices with $|\mathcal{S}| \leq t_1$, such that those t_1 intermediate variables as well as the output shares $r_{|\mathcal{O}|}$ can be perfectly simulated from $x_{|\mathcal{S}|}$.*

Proof. We show that the composition of all gadgets of Fig. 6 gives the desired result. Let $\mathcal{I}$ be a set of indices corresponding to observations of intermediate variables such that $|\mathcal{I}| \leq t_1$ and $\mathcal{O}$ be a set of output indices such that $|\mathcal{I}| + |\mathcal{O}| < d$. Consider a partition of $\mathcal{I} = \mathcal{I}^1 \cup \dots \cup \mathcal{I}^{15}$, depending on the gadget in which observation occurs (see Fig. 6).

Gadget 1 `Aff1` corresponds to line 13 in Alg. 5. Since the gadget is affine, and since by assumption $|\mathcal{I}^1| + |\mathcal{O}| \leq t_1 + |\mathcal{O}| < d$, there exist three sets of indices $\mathcal{S}_1^1, \mathcal{S}_2^1$ and $\mathcal{S}_3^1$ such that $|\mathcal{S}_1^1|, |\mathcal{S}_2^1|, |\mathcal{S}_3^1| \leq |\mathcal{I}^1| + |\mathcal{O}|$ and Gadget 1 is $((\mathcal{S}_1^1, \mathcal{S}_2^1, \mathcal{S}_3^1), (\mathcal{I}^1, \mathcal{O}))$ -NI.

Gadget 2 From first step, we have $|\mathcal{I}^2| + |\mathcal{S}_1^1| < d$. Since the gadget is t -SNI, there exists a set of indices $\mathcal{S}^2$ such that $|\mathcal{S}^2| \leq |\mathcal{I}^2|$ and Gadget 2 is $(\mathcal{S}^2, \mathcal{I}^2)$ -SNI.

Gadget 3 This gadget corresponds to the multiplication by `0xE` at line 12 of Alg. 5. By step 2, we have $|\mathcal{I}^3| + |\mathcal{S}^2| \leq |\mathcal{I}^3| + |\mathcal{I}^2| < d$. Since the gadget is affine, there exists a set of indices $\mathcal{S}^3$ such that $|\mathcal{S}^3| \leq |\mathcal{I}^3| + |\mathcal{S}^2|$ and Gadget 3 is $(\mathcal{S}^3, (\mathcal{I}^3, \mathcal{S}^2))$ -NI.

Gadget 4 From the first step, we have $|\mathcal{I}^4| + |\mathcal{S}_2^1| \leq |\mathcal{I}^4| + |\mathcal{I}^1| + |\mathcal{O}| < d$. Since the gadget is t -SNI (see Lemma 3), there exist two sets of indices $\mathcal{S}_1^4, \mathcal{S}_2^4$ such that $|\mathcal{S}_1^4|, |\mathcal{S}_2^4| \leq |\mathcal{I}^4|$ and Gadget 4 is $((\mathcal{S}_1^4, \mathcal{S}_2^4), \mathcal{I}^4)$ -SNI.

Gadget 5 This gadget corresponds to line 2 of Alg. 5. From all previous steps, we have $|\mathcal{I}^5| + |\mathcal{S}_1^4| + |\mathcal{S}^3| \leq |\mathcal{I}^5| + |\mathcal{I}^4| + |\mathcal{I}^3| + |\mathcal{I}^2| < d$. Since the gadget is t -NI, there exists a set of indices $\mathcal{S}^5$ such that $|\mathcal{S}^5| \leq |\mathcal{I}^5| + |\mathcal{S}_1^4| + |\mathcal{S}^3|$ and Gadget 5 is $(\mathcal{S}^5, (\mathcal{I}^5, (\mathcal{S}^3, \mathcal{S}_1^4)))$ -NI.

Gadget 6 This gadget corresponds to the xor operation with `0xF` at line 11 of Alg. 5. From step 4, we have $|\mathcal{I}^6| + |\mathcal{S}_2^4| \leq |\mathcal{I}^6| + |\mathcal{I}^4| < d$. Since the gadget is t -NI, there exists a set of indices $\mathcal{S}^6$ such that $|\mathcal{S}^6| \leq |\mathcal{I}^6| + |\mathcal{S}_2^4|$ and Gadget 6 is $(\mathcal{S}^6, (\mathcal{I}^6, \mathcal{S}_2^4))$ -NI.

Gadget 7 Aff_3 corresponds to line 10 of Alg. 5. From steps 6 and 1, we have $|\mathcal{I}^7| + |\mathcal{S}^6| + |\mathcal{S}_3^1| \leq |\mathcal{I}^7| + |\mathcal{I}^6| + |\mathcal{I}^4| + |\mathcal{I}^1| + |\mathcal{O}| < d$. Since Aff_3 is t -NI, there exist three sets of indices $\mathcal{S}_1^7, \mathcal{S}_2^7, \mathcal{S}_3^7$ such that $|\mathcal{S}_1^7|, |\mathcal{S}_2^7|, |\mathcal{S}_3^7| \leq |\mathcal{I}^7| + |\mathcal{S}^6| + |\mathcal{S}_3^1|$ and Gadget 7 is $((\mathcal{S}_1^7, \mathcal{S}_2^7, \mathcal{S}_3^7), (\mathcal{I}^7, (\mathcal{S}^6, \mathcal{S}_1^3)))$ -NI.

Gadget 8 From the previous step, $|\mathcal{I}^8| + |\mathcal{S}_2^7| \leq |\mathcal{I}^8| + |\mathcal{I}^7| + |\mathcal{I}^6| + |\mathcal{I}^4| + |\mathcal{I}^1| + |\mathcal{O}| < d$. Since secCompare is t -SNI (see Lemma 4), there exist two sets of indices $\mathcal{S}_1^8, \mathcal{S}_2^8$ such that $|\mathcal{S}_1^8|, |\mathcal{S}_2^8| \leq |\mathcal{I}^8|$ and Gadget 8 is $((\mathcal{S}_1^8, \mathcal{S}_2^8), \mathcal{I}^8)$ -SNI.

Gadget 9 Aff_4 corresponds to line 8 of Alg. 5. From the previous step, $|\mathcal{I}^9| + |\mathcal{S}_2^8| \leq |\mathcal{I}^9| + |\mathcal{I}^8| < d$. Since the Aff_4 is t -NI, there exist two sets of indices $\mathcal{S}_1^9$ and $\mathcal{S}_2^9$ such that $|\mathcal{S}_1^9|, |\mathcal{S}_2^9| \leq |\mathcal{I}^9| + |\mathcal{S}_2^8|$ and Gadget 9 is $((\mathcal{S}_1^9, \mathcal{S}_2^9), (\mathcal{I}^9, \mathcal{S}_2^8))$ -NI.

Gadget 10 From the previous step, $|\mathcal{I}^{10}| + |\mathcal{S}_2^9| \leq |\mathcal{I}^{10}| + |\mathcal{I}^9| + |\mathcal{I}^8| < d$. Since secAnd is t -SNI, there exist two sets of indices $\mathcal{S}_1^{10}, \mathcal{S}_2^{10}$ such that $|\mathcal{S}_1^{10}|, |\mathcal{S}_2^{10}| \leq |\mathcal{I}^{10}|$ and Gadget 10 is $((\mathcal{S}_1^{10}, \mathcal{S}_2^{10}), \mathcal{I}^{10})$ -SNI.

Gadget 11 It follows from steps 10 and 7 that $|\mathcal{I}^{11}| + |\mathcal{S}_1^{10}| + |\mathcal{S}_1^7| \leq |\mathcal{I}^{11}| + |\mathcal{I}^{10}| + |\mathcal{I}^7| + |\mathcal{I}^6| + |\mathcal{I}^4| + |\mathcal{I}^1| + |\mathcal{O}| < d$. Since the gadget is t -SNI (Lemma 4), there exist two sets of indices $\mathcal{S}_1^{11}$ and $\mathcal{S}_2^{11}$ such that $|\mathcal{S}_1^{11}|, |\mathcal{S}_2^{11}| \leq |\mathcal{I}^{11}|$ and Gadget 11 is $((\mathcal{S}_1^{11}, \mathcal{S}_2^{11}), \mathcal{I}^{11})$ -SNI.

Gadget 12 Aff_5 corresponds to line 6 of Alg. 5. From step 10, $|\mathcal{I}^{12}| + |\mathcal{S}_2^{10}| \leq |\mathcal{I}^{12}| + |\mathcal{I}^{10}| < d$. Since the gadget is t -NI, there exists a set of indices $\mathcal{S}^{12}$ such that $|\mathcal{S}^{12}| \leq |\mathcal{I}^{12}| + |\mathcal{S}_2^{10}|$ and Gadget 12 is $(\mathcal{S}^{12}, (\mathcal{I}^{12}, \mathcal{S}_2^{10}))$ -NI.

Gadget 13 Aff_6 corresponds to line 4 of Alg. 5. From Step 11, $|\mathcal{I}^{13}| + |\mathcal{S}_2^{11}| \leq |\mathcal{I}^{13}| + |\mathcal{I}^{11}| < d$. Since the gadget is t -NI, there exists a set of indices $\mathcal{S}^{13}$ such that $|\mathcal{S}^{13}| \leq |\mathcal{I}^{13}| + |\mathcal{S}_2^{11}|$ and Gadget 13 is $(\mathcal{S}^{13}, (\mathcal{I}^{13}, \mathcal{S}_2^{11}))$ -NI.

Gadget 14 It follows from steps 13, 12, 9 and 7 that $|\mathcal{I}^{14}| + |\mathcal{S}^{13}| + |\mathcal{S}^{12}| + |\mathcal{S}_2^9| + |\mathcal{S}_3^7| \leq |\mathcal{I}^{14}| + |\mathcal{I}^{13}| + |\mathcal{I}^{12}| + |\mathcal{I}^{11}| + |\mathcal{I}^{10}| + |\mathcal{I}^9| + |\mathcal{I}^8| + |\mathcal{I}^7| + |\mathcal{I}^6| + |\mathcal{I}^4| + |\mathcal{I}^1| + |\mathcal{O}| < d$. Since secCompare is t -SNI (its second input is a public sharing of a constant), there exists a set of indices $\mathcal{S}^{14}$ such that $|\mathcal{S}^{14}| \leq |\mathcal{I}^{14}|$ and Gadget 14 is $(\mathcal{S}^{14}, \mathcal{I}^{14})$ -SNI.

Gadget 15 This gadget corresponds to line 1 of Alg. 5. From steps 14, 11 and 8, $|\mathcal{I}^{15}| + |\mathcal{S}^{14}| + |\mathcal{S}_1^{11}| + |\mathcal{S}_1^8| \leq |\mathcal{I}^{15}| + |\mathcal{I}^{14}| + |\mathcal{I}^{11}| + |\mathcal{I}^8| < d$. Since the gadget is affine, there exists a set of indices $\mathcal{S}^{15}$ such that $|\mathcal{S}^{15}| \leq |\mathcal{I}^{15}| + |\mathcal{S}^{14}| + |\mathcal{S}_1^{11}| + |\mathcal{S}_1^8|$ and Gadget 15 is $(\mathcal{S}^{15}, (\mathcal{I}^{15}, (\mathcal{S}^{14}, \mathcal{S}_1^{11}, \mathcal{S}_1^8)))$ -NI.

Each of the previous steps ensures the existence of a simulator for each gadget. Let $\mathcal{S} = \mathcal{S}^{15} \cup \mathcal{S}^5$, we can then compose these simulators to perfectly simulate the computation of the composition from $x_{|\mathcal{S}|}$. Furthermore, from steps 15 and 5 we know that $|\mathcal{S}| \leq |\mathcal{I}^{15}| + |\mathcal{I}^{14}| + |\mathcal{I}^{11}| + |\mathcal{I}^8| + |\mathcal{I}^5| + |\mathcal{I}^4| + |\mathcal{I}^3| + |\mathcal{I}^2| \leq t_1$, which concludes the proof. $\square$

3.2.5 Improvement Assessment

To estimate the performance improvement of our method, we compare the number of calls to standard gadgets (secAnd , secXor , secShift and refresh) required by Alg. 5 with that of [GG25, Alg. 5]. In both cases, the secure addition relies on the KoggeStone gadget presented in Alg. 3.

Table 3 shows that Alg. 5 makes noticeably fewer calls to standard gadgets compared to [GG25, Alg. 5]. This improvement mainly comes from the use of the Kogge-Stone adder, which makes up most of the gadget calls (requiring at most 8 secAnd s, 5 secXor s, 3 refresh calls and 7 secShift s, each time it is invoked). Our implementation calls the Kogge-Stone adder only 3 times, whereas the approach in [GG25, Alg. 5] calls it 6 times, cutting down this expensive part by half.

4 Implementation

We validate the correctness and evaluate the performance of our masked Gaussian sampler across the three FrodoKEM parameter sets on an ARM Cortex-M3 microcontroller which

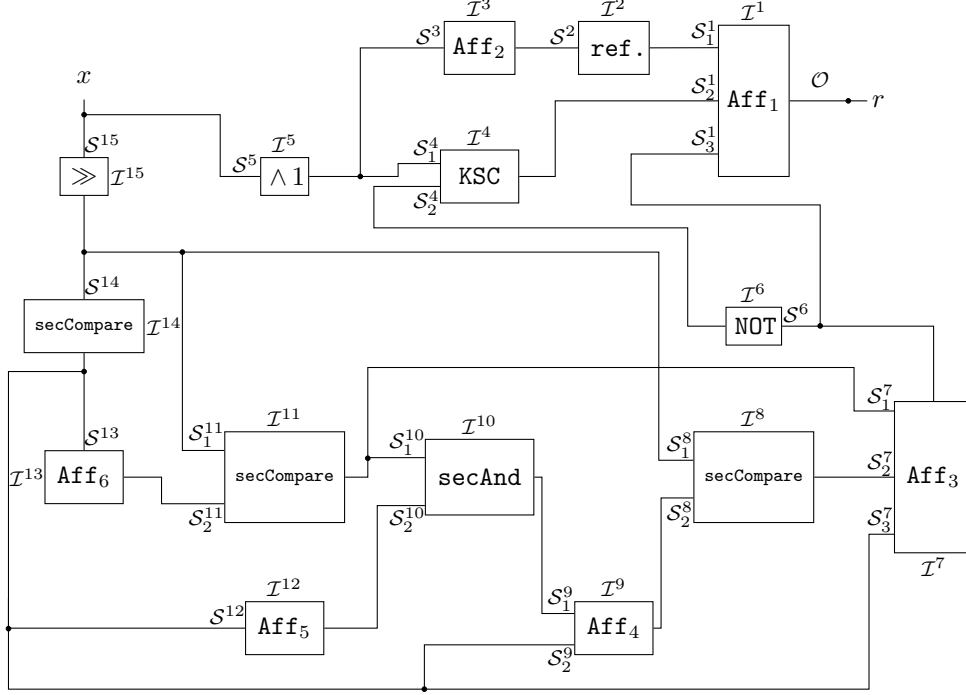


Figure 6: FrodoKEM-1344 Sampler gadget

Table 3: Number of calls to main gadgets in Algs. 7 and 5 compared with [GG25, Alg. 5].

	Version	FrodoKEM-640		FrodoKEM-976		FrodoKEM-1344	
		[GG25]	Alg. 7	[GG25]	Alg. 7	[GG25]	Alg. 5
Gadget	secAnd	232	42	196	42	124	29
	secShift	207	50	175	50	111	37
	secXor	247	41	209	41	133	29
	refresh	12	15	10	15	6	11

is a representative platform for embedded systems where SCA is especially important. Tables 4 and 5 show the number of cycles required to generate one sample, along with the number of 16-bit randoms needed to be generated per call. In both cases, we directly compare our design with the masked implementation from [GG25] available on https://github.com/fragerar/masked_Frodo, which serves as the current baseline. Since our implementation deals with Boolean shares only, we remove the Boolean to Arithmetic conversion from [GG25] for our comparisons.

First-Order Implementation. Table 4 shows our benchmarking results for order one, with the entire implementation written in assembly for the ARM Cortex-M3. All building blocks of the reference implementation from [GG25] are also written in assembly, making the comparison fair. Our sampler achieves over 50x speedup across all FrodoKEM parameter sets and uses more than 30x fewer 16-bit random values. To ensure consistency between the two implementations, random values are pre-generated and stored in a buffer, so the reported timings account only for table lookups. Note that the memory consumption of the Gaussian sampler is very small, on the order of a few dozen of bytes. The CDT constants are hardcoded, so the memory usage is limited to standard local variables. This overhead is similar to that of [GG25] and remains negligible compared to the full masked

FrodoKEM implementation, which requires over 100 kB (see [GG25, Table 7]).

Table 4: Cycle count on ARM Cortex M3 and number of calls to `rand16` for first order Gaussian Sampler (Algs. 7 and 5), compared with [GG25, Alg. 5].

	FrodoKEM-640		FrodoKEM-976		FrodoKEM-1344	
Number of	cycles	rands	cycles	rands	cycles	rands
[GG25]	5 590	244	4 780	206	3 160	130
This paper (Algs. 7 and 5)	800	57	800	57	557	40

High-order implementation. Table 5 shows our benchmarking results for higher-order masking, up to order 8. These results are obtained from a generic C implementation except for the two gadgets `secAnd` and `refresh` which are implemented in assembly on Cortex-M3. As before, all random values are pre-generated and stored in memory, so the reported timings account only for table lookups.

Table 5: Number of cycles on ARM Cortex M3 and number of calls to `rand16` for order ≤ 8 Gaussian Sampler (Algs. 7 and 5), for the three versions of FrodoKEM.

	order	2	3	4	5	6	7	8
FrodoKEM-640	# cycles	6216	9366	13180	17738	23246	29254	36339
FrodoKEM-976	# rands	171	342	570	855	1197	1596	2052
FrodoKEM-1344	# cycles	4370	6570	9233	12412	16249	20432	25363
	# rands	120	240	400	600	840	1120	1440

5 Experimental Evaluation of the Sampler

To assess the side-channel resistance of our approach, we performed a standard first-order Test Vector Leakage Assessment (TVLA) on the FrodoKEM-1344 Gaussian sampler running on an ARM Cortex-M3 microcontroller. The implementation is written in assembly without data-dependent branches or memory accesses to achieve constant time, as confirmed by the stable execution profile observed in the traces.

We adopted the “fixed vs random” test setting. Two sets of traces were generated: in the fixed set, all inputs were set to a fixed value, whereas in the random set, the inputs were freshly generated using the same on-device PRNG for both the unmasked and masked implementations. Apart from masking, both configurations used identical test vectors and identical randomness streams. Likewise, the same physical chip, measurement setup, and acquisition parameters were used throughout all experiments, ensuring comparability between the masked and unmasked cases.

Capture setup, triggering and alignment. Measurements were acquired using a digital oscilloscope at a sampling rate of 100 MS/s. The sampler code was instrumented with GPIO toggles bracketing the region of interest, enabling reliable selection of the capture window with very limited desynchronization. Trace alignment was performed by synchronizing on a distinctive peak within a 60-sample window, and no additional preprocessing (such as filtering, resampling, or denoising) was applied.

Test-vector selection. For the “fixed vs random” TVLA, each acquisition batch consisted of 50 % fixed and 50 % random test vectors. The fixed test vectors were selected at random once and then reused for both masked and unmasked measurements.

***t*-test procedure.** At each time sample, a Welch's *t*-test was computed and compared to the standard ± 4.5 threshold for first-order leakage detection. In the unmasked configuration, significant leakage appears clearly (Fig. 7a), with threshold crossings after as few as 20k traces. In the masked configuration, all *t*-values remain within the ± 4.5 interval even after 200k traces (Fig. 7b). This indicates that masking successfully suppresses observable first-order leakage.

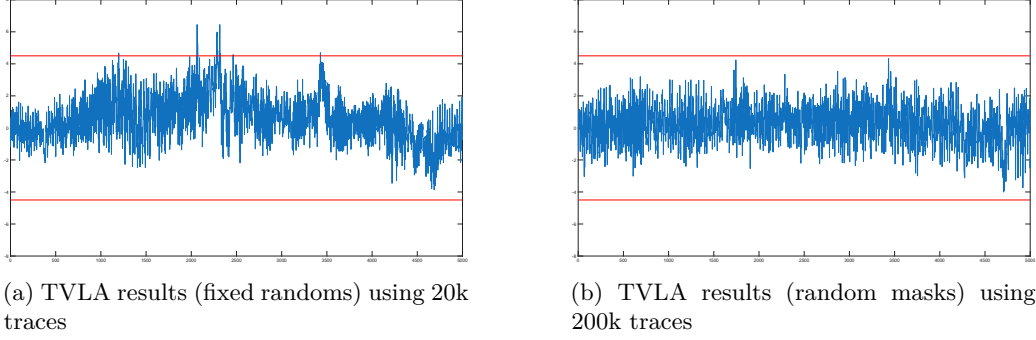


Figure 7: Side-channel evaluation of the masked FrodoKEM-1344 Gaussian sampler under fixed vs. random mask conditions (the ± 4.5 threshold is marked by red lines).

6 Conclusion and Future Work

In this work, we introduced a new high-order masking gadget for the Gaussian sampler of FrodoKEM, built around a binary decision tree structure that significantly reduces the number of secure comparisons. Since these comparisons are the main performance bottleneck in masked CDT-based sampling, our approach provides a substantial speed improvement while preserving full constant-time behavior. We also introduced a new secure comparison gadget based on a modified KoggeStone adder and proved its *t*-SNI security, which forms the foundation for the security of our full sampler.

Our implementation on an ARM Cortex-M3 demonstrates substantial performance gains, achieving around 82% reduction in cycle count and a 69% reduction in random number usage compared to the previous state of the art, and only require a few tens of bytes of working memory.

As further work, our approach could be extended to other lattice-based schemes that rely on CDT-based Gaussian sampling, such as qTESLA [ABB⁺20], HAWK [BBD⁺25], or Falcon [Fal20]. In qTESLA, the discrete Gaussian distribution is generated using a CDT with 77 or 111 entries depending on the security level, with 64-bit entries for level I and 128-bit entries for level III, which could directly benefit from our decision tree optimization. HAWK similarly employs CDT-based sampling for its noise generation, using two distinct CDTs with 10 to 13 entries each and 80-bit thresholds. Finally, Falcon's Gaussian sampler uses a Reverse Cumulative Distribution Table (RCDT), which functions analogously to a CDT but stores reverse cumulative probabilities; this design could also benefit from our binary decision tree optimization.

Acknowledgments

The authors gratefully thank their colleague Guillaume Barbu for carrying out the measurements described in Sect. 5 and for generating the associated figures.

Appendix A Masking the CDT sampler for FrodoKEM-640 and FrodoKEM-976

Both FrodoKEM-640 and FrodoKEM-976 use the sampling procedure described in Alg. 1, with their own respective cumulative distribution tables:

$$T_{\chi_{640}} = (4643, 13363, 20579, 25843, 29227, 31145, 32103, 32525, 32689, 32745, 32762, 32766, 32767)$$

$$T_{\chi_{976}} = (5638, 15915, 23689, 28571, 31116, 32217, 32613, 32731, 32760, 32766, 32767)$$

To obtain a constant-time implementation based on a binary decision tree, the CDTs must be padded so that their lengths matches the form $2^m - 1$. For FrodoKEM-640, we pad the table with two additional occurrences of the value 32767 to extend the table to length 15. Similarly, for FrodoKEM-976, four values 32767 are appended to the CDT to reach the same length.

This gives Fig. 8, which shows that the number of integer comparisons is reduced to only 4, compared to 12 and 10 in the original implementations of FrodoKEM-640 and FrodoKEM-976, respectively. The corresponding constant-time algorithm is presented in Alg. 6, with its masked version provided in Alg. 7. Both rely on a set of constants specific to each parameter set, which are defined in Table 6.

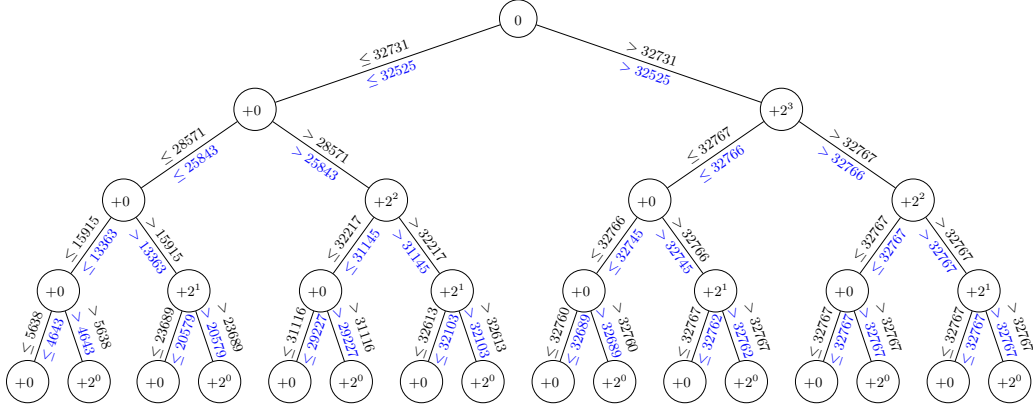


Figure 8: Binary trees associated with the Gaussian Samplers of FrodoKEM-640 (in blue) and FrodoKEM-976 (in black)

Table 6: Constants for GaussianSample-640/976 and secGaussianSample-640/976 (Algs. 6 and 7).

	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8
FrodoKEM-640	32525	6925	25843	19852	19866	19418	13363	19783
FrodoKEM-976	32731	4196	28571	17395	17394	16853	15915	19553

	C_9	C_{10}	C_{11}	C_{12}	C_{13}	C_{14}	C_{15}
FrodoKEM-640	19724	16907	16960	24646	24584	28050	4643
FrodoKEM-976	19558	19080	19087	28557	28554	27134	5638

Algorithm 6 GaussianSample-640/976(x). Gaussian Sampler for FrodoKEM-640/976**Input:** A 16-bit integer x .**Output:** GaussianSample(x)

- 1: $a_0 \leftarrow x \gg 1$
- 2: $b_0 \leftarrow (a_0 > C_1)$
- 3: $a_1 \leftarrow (C_2 \cdot b_0) \oplus C_3$
- 4: $b_1 \leftarrow (a_0 > a_1)$
- 5: $a_2 \leftarrow (b_1 \cdot ((b_0 \cdot C_4) \oplus C_5)) \oplus (b_0 \cdot C_6) \oplus C_7$
- 6: $b_2 \leftarrow (a_0 > a_2)$
- 7: $a_3 \leftarrow (b_2 \cdot (b_1 \cdot ((b_0 \cdot C_8) \oplus C_9) \oplus (b_0 \cdot C_{10}) \oplus C_{11})) \oplus (b_1 \cdot ((b_0 \cdot C_{12}) \oplus C_{13})) \oplus (b_0 \cdot C_{14}) \oplus C_{15}$
- 8: $b_3 \leftarrow (a_0 > a_3)$
- 9: **return** $(-1)^{x \wedge 1} \cdot ((b_0 \ll 3) \oplus (b_1 \ll 2) \oplus (b_2 \ll 1) \oplus b_3)$

Algorithm 7 secGaussianSample-640/976($(x_i)_{1 \leq i \leq d}$). Masked Gaussian Sampler for FrodoKEM-640/976**Input:** Boolean shares $(x_i)_{1 \leq i \leq d}$ of a 16-bit integer x .**Output:** Boolean shares $(r_i)_{1 \leq i \leq d}$ of GaussianSample(x).

- 1: $(a_i^{(0)}) \leftarrow (x_i \gg 1)_{1 \leq i \leq d}$
- 2: $(s_i)_{1 \leq i \leq d} \leftarrow (x_i \wedge 1)_{1 \leq i \leq d}$
- 3: $(b_i^{(0)})_{1 \leq i \leq d} \leftarrow \text{secCompare}((a_i^{(0)})_{1 \leq i \leq d}, (C_1, 0, \dots, 0))$
- 4: $(a_i^{(1)})_{1 \leq i \leq d} \leftarrow ((b_i^{(0)} \cdot C_2))_{1 \leq i \leq d} \oplus (C_3, 0, \dots, 0)$
- 5: $(b_i^{(1)})_{1 \leq i \leq d} \leftarrow \text{secCompare}((a_i^{(0)})_{1 \leq i \leq d}, (a_i^{(1)})_{1 \leq i \leq d})$
- 6: $(a_i^{(2)})_{1 \leq i \leq d} \leftarrow ((b_i^{(0)} \cdot C_4))_{1 \leq i \leq d} \oplus (C_5, 0, \dots, 0)$
- 7: $(a_i^{(2)})_{1 \leq i \leq d} \leftarrow \text{secAnd}((a_i^{(2)})_{1 \leq i \leq d}, (b_i^{(1)} \cdot 0xFFFF)_{1 \leq i \leq d})$
- 8: $(a_i^{(2)})_{1 \leq i \leq d} \leftarrow \text{secXor}((b_i^{(0)} \cdot C_6)_{1 \leq i \leq d}, (a_i^{(2)})_{1 \leq i \leq d}) \oplus (C_7, 0, \dots, 0)$
- 9: $(b_i^{(2)})_{1 \leq i \leq d} \leftarrow \text{secCompare}((a_i^{(0)})_{1 \leq i \leq d}, (a_i^{(2)})_{1 \leq i \leq d})$
- 10: $(a_i^{(3)})_{1 \leq i \leq d} \leftarrow ((b_i^{(0)} \cdot C_8))_{1 \leq i \leq d} \oplus (C_9, 0, \dots, 0)$
- 11: $(a_i^{(3)})_{1 \leq i \leq d} \leftarrow \text{secAnd}((a_i^{(3)})_{1 \leq i \leq d}, (b_i^{(1)} \cdot 0xFFFF)_{1 \leq i \leq d})$
- 12: $(a_i^{(3)})_{1 \leq i \leq d} \leftarrow \text{secXor}((b_i^{(0)} \cdot C_{10})_{1 \leq i \leq d}, (a_i^{(3)})_{1 \leq i \leq d}) \oplus (C_{11}, 0, \dots, 0)$
- 13: $(a_i^{(3)})_{1 \leq i \leq d} \leftarrow \text{secAnd}((a_i^{(3)})_{1 \leq i \leq d}, (b_i^{(2)} \cdot 0xFFFF)_{1 \leq i \leq d})$
- 14: $(a_i^{(4)})_{1 \leq i \leq d} \leftarrow ((b_i^{(0)} \cdot C_{12}))_{1 \leq i \leq d} \oplus (C_{13}, 0, \dots, 0)$
- 15: $(a_i^{(4)})_{1 \leq i \leq d} \leftarrow \text{secAnd}((a_i^{(4)})_{1 \leq i \leq d}, (b_i^{(1)} \cdot 0xFFFF)_{1 \leq i \leq d})$
- 16: $(a_i^{(3)})_{1 \leq i \leq d} \leftarrow \text{secXor}(\text{secXor}((b_i^{(0)} \cdot C_{14})_i, (a_i^{(3)})_i), (a_i^{(4)})_i) \oplus (C_{15}, 0, \dots, 0)$
- 17: $(b_i^{(3)})_{1 \leq i \leq d} \leftarrow \text{secCompare}((a_i^{(0)})_{1 \leq i \leq d}, (a_i^{(3)})_{1 \leq i \leq d})$
- 18: $(r_i)_{1 \leq i \leq d} \leftarrow \text{secXor}(\text{secXor}(\text{secXor}((b_i^{(0)} \ll 3)_i, (b_i^{(1)} \ll 2)_i), (b_i^{(2)} \ll 1)_i), (b_i^{(3)}))$
- 19: $(g_i)_{1 \leq i \leq d} \leftarrow \text{KoggeStoneCarry}((r_i)_{1 \leq i \leq d} \oplus (0x1F, 0, \dots, 0), (s_i)_{1 \leq i \leq d}, 5)$
- 20: $(s_i)_{1 \leq i \leq d} \leftarrow \text{refresh}((s_i \cdot 0x1E)_{1 \leq i \leq d}, 5)$
- 21: $(r_i)_{1 \leq i \leq d} \leftarrow \text{secXor}(\text{secXor}((r_i)_{1 \leq i \leq d}, (g_i \ll 1)_{1 \leq i \leq d}), (s_i)_{1 \leq i \leq d})$
- 22: **return** $(r_i)_{1 \leq i \leq d}$

Theorem 2 (t -SNI of secGaussianSample-640/976 (Alg. 7)). *Let $(x_i)_{1 \leq i \leq d}$ be the input shares of the secGaussianSample-640/976 gadget, and let $(r_i)_{1 \leq i \leq d}$ be the output shares. For any set of t_1 intermediate variables and any subset $\mathcal{O}$ of output indices such that $t_1 + |\mathcal{O}| < d$, there exists a subset $\mathcal{S}$ of indices with $|\mathcal{S}| \leq t_1$, such that those t_1 intermediate variables as well as the output shares $r_{i \in \mathcal{O}}$ can be perfectly simulated from $x_{i \in \mathcal{S}}$.*

Proof. We prove Theorem 2 by adapting proof of Theorem 1. Indeed, Alg. 7 has a large

common part with Alg. 5; the main difference lies in the depth of the decision tree: FrodoKEM-640 and FrodoKEM-976 use a tree that is one level deeper than FrodoKEM-1344. More precisely, Line 10 of Alg. 5 is replaced by lines 10-18 in Alg. 7 while all the rest is common, apart from the parameter-specific constants. From this observation, we isolate all new gadgets that are considered in lines 10-18, and revisit the proof of Theorem 1 by updating all components of Fig. 6 that are affected by the additional level in the decision tree. We highlight those impacted gadgets in Fig. 9, and we show that the composition with the new gadgets still provides the t -SNI property of the whole Alg. 7.

Let $\mathcal{I}$ be a set of indices corresponding to observations of intermediate variables such that $|\mathcal{I}| \leq t_1$ and $\mathcal{O}$ be a set of output indices such that $|\mathcal{I}| + |\mathcal{O}| < d$. Consider a partition of $\mathcal{I} = \bigcup_{i=1}^{23} \mathcal{I}^i$, depending on the gadget in which observation occurs (see Fig. 9).

Gadgets 1-6, 9-10, 12-13 (see Fig. 6) Those gadgets remain exactly the same as in the proof Theorem 1, and their simulation is also performed identically (for this reason, they do not appear on Fig. 9).

Gadget 7 G7 corresponds to line 18 of Alg. 7. The condition $|\mathcal{I}^7| + |\mathcal{S}^6| + |\mathcal{S}_3^1| \leq t_1 + |\mathcal{O}| < d$ is still valid from steps 6 and 1 (proof of Theorem 1). Since the G7 gadget is t -NI, there exist four sets of indices $\mathcal{S}_1^7, \mathcal{S}_2^7, \mathcal{S}_3^7, \mathcal{S}_4^7$, such that $|\mathcal{S}_1^7|, |\mathcal{S}_2^7|, |\mathcal{S}_3^7|, |\mathcal{S}_4^7| \leq |\mathcal{I}^7| + |\mathcal{S}^6| + |\mathcal{S}_3^1|$, and Gadget 7 is $((\mathcal{S}_1^7, \mathcal{S}_2^7, \mathcal{S}_3^7, \mathcal{S}_4^7), (\mathcal{I}^7, (\mathcal{S}^6, \mathcal{S}_3^1)))$ -NI.

Gadget 16 From the previous step, $|\mathcal{I}^{16}| + |\mathcal{S}_4^7| \leq |\mathcal{I}^{16}| + |\mathcal{I}^7| + |\mathcal{I}^6| + |\mathcal{I}^4| + |\mathcal{I}^1| + |\mathcal{O}| \leq t_1 + |\mathcal{O}| < d$. Since **secCompare** is t -SNI (Lemma 4), there exist two sets of indices $\mathcal{S}_1^{16}, \mathcal{S}_2^{16}$ such that $|\mathcal{S}_1^{16}|, |\mathcal{S}_2^{16}| \leq |\mathcal{I}^{16}|$ and Gadget 16 is $((\mathcal{S}_1^{16}, \mathcal{S}_2^{16}), \mathcal{I}^{16})$ -SNI.

Gadget 17 Aff₇ corresponds to line 16 of Alg. 7. From previous step, $|\mathcal{I}^{17}| + |\mathcal{S}_1^{16}| \leq |\mathcal{I}^{17}| + |\mathcal{I}^{16}| \leq t_1 < d$. Since Aff₇ is affine and thus t -NI, there exist three sets of indices $\mathcal{S}_1^{17}, \mathcal{S}_2^{17}, \mathcal{S}_3^{17}$ such that $|\mathcal{S}_1^{17}|, |\mathcal{S}_2^{17}|, |\mathcal{S}_3^{17}| \leq |\mathcal{I}^{17}| + |\mathcal{S}_1^{16}|$ and Gadget 17 is $((\mathcal{S}_1^{17}, \mathcal{S}_2^{17}, \mathcal{S}_3^{17}), (\mathcal{I}^{17}, \mathcal{S}_1^{16}))$ -NI.

Gadget 18 This gadget corresponds to line 15 of Alg. 7. From previous step, $|\mathcal{I}^{18}| + |\mathcal{S}_2^{17}| \leq |\mathcal{I}^{18}| + |\mathcal{I}^{17}| + |\mathcal{I}^{16}| \leq t_1 < d$. Since **secAnd** is t -SNI, there exist two sets of indices $\mathcal{S}_1^{18}$ and $\mathcal{S}_2^{18}$ such that $|\mathcal{S}_1^{18}|, |\mathcal{S}_2^{18}| \leq |\mathcal{I}^{18}|$ and Gadget 18 is $((\mathcal{S}_1^{18}, \mathcal{S}_2^{18}), \mathcal{I}^{18})$ -SNI.

Gadget 19 Aff₈ corresponds to line 14 of Alg. 7. From previous step, $|\mathcal{I}^{19}| + |\mathcal{S}_2^{18}| \leq |\mathcal{I}^{19}| + |\mathcal{I}^{18}| \leq t_1 < d$. Since the gadget is t -NI, there exists a set of indices $\mathcal{S}^{19}$ such that $|\mathcal{S}^{19}| \leq |\mathcal{I}^{19}| + |\mathcal{S}_2^{18}|$ and Gadget 19 is $(\mathcal{S}^{19}, (\mathcal{I}^{19}, \mathcal{S}_2^{18}))$ -NI.

Gadget 20 This gadget corresponds to line 13 of Alg. 7. From step 17, $|\mathcal{I}^{20}| + |\mathcal{S}_1^{17}| \leq |\mathcal{I}^{20}| + |\mathcal{I}^{17}| + |\mathcal{I}^{16}| \leq t_1 < d$. Since **secAnd** is t -SNI, there exist two sets of indices $\mathcal{S}_1^{20}$ and $\mathcal{S}_2^{20}$ such that $|\mathcal{S}_1^{20}|, |\mathcal{S}_2^{20}| \leq |\mathcal{I}^{20}|$ and Gadget 20 is $((\mathcal{S}_1^{20}, \mathcal{S}_2^{20}), \mathcal{I}^{20})$ -SNI.

Gadget 21 Aff₉ corresponds to line 12 of Alg. 7. From previous step, $|\mathcal{I}^{21}| + |\mathcal{S}_2^{20}| \leq |\mathcal{I}^{21}| + |\mathcal{I}^{20}| \leq t_1 < d$. Since the gadget is t -NI, there exist two sets of indices $\mathcal{S}_1^{21}$ and $\mathcal{S}_2^{21}$ such that $|\mathcal{S}_1^{21}|, |\mathcal{S}_2^{21}| \leq |\mathcal{I}^{21}| + |\mathcal{S}_2^{20}|$ and Gadget 21 is $((\mathcal{S}_1^{21}, \mathcal{S}_2^{21}), (\mathcal{I}^{21}, \mathcal{S}_2^{20}))$ -NI.

Gadget 22 This gadget corresponds to line 11 of Alg. 7. From previous step, $|\mathcal{I}^{22}| + |\mathcal{S}_1^{21}| \leq |\mathcal{I}^{22}| + |\mathcal{I}^{21}| + |\mathcal{I}^{20}| \leq t_1 < d$. Since **secAnd** is t -SNI, there exist two sets of indices $\mathcal{S}_1^{22}$ and $\mathcal{S}_2^{22}$ such that $|\mathcal{S}_1^{22}|, |\mathcal{S}_2^{22}| \leq |\mathcal{I}^{22}|$ and Gadget 22 is $((\mathcal{S}_1^{22}, \mathcal{S}_2^{22}), \mathcal{I}^{22})$ -SNI.

Gadget 23 Aff₁₀ corresponds to line 10 of Alg. 7. From previous step, $|\mathcal{I}^{23}| + |\mathcal{S}_2^{22}| \leq |\mathcal{I}^{23}| + |\mathcal{I}^{22}| \leq t_1 < d$. Since the gadget is t -NI, there exists a set of indices $\mathcal{S}^{23}$ such that $|\mathcal{S}^{23}| \leq |\mathcal{I}^{23}| + |\mathcal{S}_2^{22}|$ and Gadget 23 is $(\mathcal{S}^{23}, (\mathcal{I}^{23}, \mathcal{S}_2^{22}))$ -NI.

We now end with the update of the remaining gadgets 8, 11, 14 and 15.

Gadget 8 G8 corresponds to line 9 of Alg. 7. From steps 7 and 20, $|\mathcal{I}^8| + |\mathcal{S}_2^7| + |\mathcal{S}_1^{20}| \leq |\mathcal{I}^{20}| + |\mathcal{I}^8| + |\mathcal{I}^7| + |\mathcal{I}^6| + |\mathcal{I}^4| + |\mathcal{I}^1| + |\mathcal{O}| \leq t_1 + |\mathcal{O}| < d$. Since **secCompare** is t -SNI, there exist two sets of indices $\mathcal{S}_1^8, \mathcal{S}_2^8$ such that $|\mathcal{S}_1^8|, |\mathcal{S}_2^8| \leq |\mathcal{I}^8|$ and Gadget 8 is $((\mathcal{S}_1^8, \mathcal{S}_2^8), \mathcal{I}^8)$ -SNI.

Gadget 11 G11 corresponds to line 5 of Alg. 7. From steps 10, 7, 18 and 22, $|\mathcal{I}^{11}| + |\mathcal{S}_1^{10}| + |\mathcal{S}_1^7| + |\mathcal{S}_1^{18}| + |\mathcal{S}_1^{22}| \leq |\mathcal{I}^{22}| + |\mathcal{I}^{18}| + |\mathcal{I}^{11}| + |\mathcal{I}^{10}| + |\mathcal{I}^7| + |\mathcal{I}^6| + |\mathcal{I}^4| + |\mathcal{I}^1| + |\mathcal{O}| \leq t_1 + |\mathcal{O}| < d$. Since **secCompare** is t -SNI (Lemma 4), there exist two sets of indices $\mathcal{S}_1^{11}$ and $\mathcal{S}_2^{11}$ such

that $|\mathcal{S}_1^{11}|, |\mathcal{S}_2^{11}| \leq |\mathcal{I}^{11}|$ and Gadget 11 is $((\mathcal{S}_1^{11}, \mathcal{S}_2^{11}), \mathcal{I}^{11})$ -SNI.

Gadget 14 G14 corresponds to line 3 of Alg. 7. From steps 13, 12, 9, 7, 17, 19, 21 and 23, $|\mathcal{I}^{14}| + |\mathcal{S}^{13}| + |\mathcal{S}^{12}| + |\mathcal{S}_2^9| + |\mathcal{S}_3^7| + |\mathcal{S}^{23}| + |\mathcal{S}_2^{21}| + |\mathcal{S}^{19}| + |\mathcal{S}_3^{17}| \leq |\mathcal{I}^{23}| + |\mathcal{I}^{22}| + |\mathcal{I}^{21}| + |\mathcal{I}^{20}| + |\mathcal{I}^{19}| + |\mathcal{I}^{18}| + |\mathcal{I}^{14}| + |\mathcal{I}^{13}| + |\mathcal{I}^{12}| + |\mathcal{I}^{11}| + |\mathcal{I}^{10}| + |\mathcal{I}^9| + |\mathcal{I}^8| + |\mathcal{I}^7| + |\mathcal{I}^6| + |\mathcal{I}^4| + |\mathcal{I}^1| + |\mathcal{O}| \leq t_1 + |\mathcal{O}| < d$. Since **secCompare** is t -SNI (its second input is here a public sharing of a constant), there exists a set of indices $\mathcal{S}^{14}$ such that $|\mathcal{S}^{14}| \leq |\mathcal{I}^{14}|$ and Gadget 14 is $(\mathcal{S}^{14}, \mathcal{I}^{14})$ -SNI.

Gadget 15 G15 corresponds to line 1 of Alg. 7. From steps 14, 11, 8 and 16, $|\mathcal{I}^{15}| + |\mathcal{S}^{14}| + |\mathcal{S}_1^{11}| + |\mathcal{S}_1^8| + |\mathcal{S}_2^{16}| \leq |\mathcal{I}^{16}| + |\mathcal{I}^{15}| + |\mathcal{I}^{14}| + |\mathcal{I}^{11}| + |\mathcal{I}^8| \leq t_1 < d$. Since the gadget is t -NI (affine), there exists a set of indices $\mathcal{S}^{15}$ such that $|\mathcal{S}^{15}| \leq |\mathcal{I}^{15}| + |\mathcal{S}^{14}| + |\mathcal{S}_1^{11}| + |\mathcal{S}_1^8| + |\mathcal{S}_2^{16}|$ and Gadget 15 is $(\mathcal{S}^{15}, (\mathcal{I}^{15}, (\mathcal{S}^{14}, \mathcal{S}_1^{11}, \mathcal{S}_1^8, \mathcal{S}_2^{16})))$ -NI.

Each of the previous steps ensures the existence of a simulator for each gadget. Let $\mathcal{S} = \mathcal{S}^{15} \cup \mathcal{S}^5$, we can then compose these simulators to perfectly simulate the computation of the composition from $x|_{\mathcal{S}}$. Furthermore, from steps 15 and 5 we know that $|\mathcal{S}| \leq |\mathcal{S}^{15}| + |\mathcal{S}^5| \leq |\mathcal{I}^{16}| + |\mathcal{I}^{15}| + |\mathcal{I}^{14}| + |\mathcal{I}^{11}| + |\mathcal{I}^8| + |\mathcal{I}^5| + |\mathcal{I}^4| + |\mathcal{I}^3| + |\mathcal{I}^2| \leq t_1$, which concludes the proof. $\square$

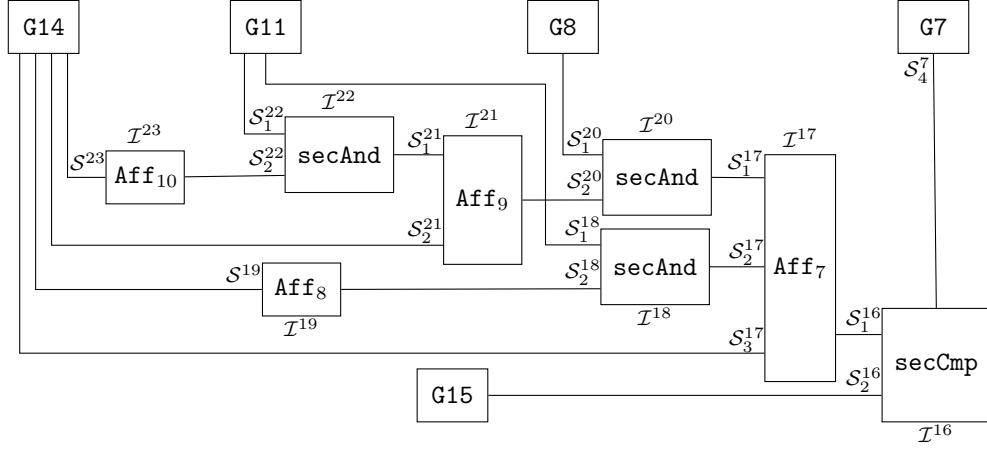


Figure 9: FrodoKEM-640/976 Sampler gadget

References

- [ABB⁺20] Erdem Alkim, Paulo S. L. M. Barreto, Nina Bindel, Juliane Krämer, Patrick Longa, and Jefferson E. Ricardini. The Lattice-Based Digital Signature Scheme qTESLA. In Mauro Conti, Jianying Zhou, Emiliano Casalicchio, and Angelo Spognardi, editors, *Applied Cryptography and Network Security*, pages 441–460, Cham, 2020. Springer International Publishing.
- [ABD⁺23] Erdem Alkim, Joppe W. Bos, Léo Ducas, Karen Easterbrook, Brian LaMacchia, Patrick Longa, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Chris Peikert, Ananth Raghunathan, and Douglas Stebila. FrodoKEM: Learning With Errors Key Encapsulation. Submission to the NIST Post-Quantum Standardization project, 2023. <https://frodokem.org/>.
- [ANS23] ANSSI views on the Post-Quantum Cryptography transition (2023 follow up), 2023. https://cyber.gouv.fr/sites/default/files/document/follow_up_position_paper_on_post_quantum_cryptography.pdf.
- [BBD⁺15a] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, and Pierre-Yves Strub. Verified Proofs of Higher-Order Masking. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015*, pages 457–485, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
- [BBD⁺15b] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Compositional Verification of Higher-Order Masking: Application to a Verifying Masking Compiler. Cryptology ePrint Archive, Paper 2015/506, revision 2015-05-27, 2015. <https://eprint.iacr.org/2015/506/20150527:192221>.
- [BBD⁺16] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Strong Non-Interference and Type-Directed Higher-Order Masking. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’16, page 116129, New York, NY, USA, 2016. Association for Computing Machinery.
- [BBD⁺25] Joppe W. Bos, Olivier Bronchain, Léo Ducas, Serge Fehr, Yu-Hsuan Huang, Thomas Pornin, Eamonn W. Postlethwaite, Thomas Prest, Ludo N. Pulles, and Wessel van Woerden. HAWK. <https://hawk-sign.info/>, 2025. Version 2025-02-05.
- [BBE⁺18] Gilles Barthe, Sonia Belaïd, Thomas Espitau, Pierre-Alain Fouque, Benjamin Grégoire, Mélissa Rossi, and Mehdi Tibouchi. Masking the GLP Lattice-Based Signature Scheme at Any Order. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, pages 354–384, Cham, 2018. Springer International Publishing.
- [BCD⁺16] Joppe Bos, Craig Costello, Leo Ducas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. Frodo: Take off the Ring! Practical, Quantum-Secure Key Exchange from LWE. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’16, page 10061018, New York, NY, USA, 2016. Association for Computing Machinery.
- [Ber09] Daniel J. Bernstein. Introduction to Post-Quantum Cryptography. *Post-Quantum Cryptography*, pages 1–14, 2009.

- [BSI21] Migration to Post Quantum Cryptography, recommendations for action by the BSI, 2021. https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Crypto/Migration_to_Post_Quantum_Cryptography.pdf.
- [CGV14] Jean-Sébastien Coron, Johann Großschädl, and Praveen Kumar Vadnala. Secure conversion between boolean and arithmetic masking of any order. In Lejla Batina and Matthew Robshaw, editors, *Cryptographic Hardware and Embedded Systems – CHES 2014*, pages 188–205, Berlin, Heidelberg, 2014. Springer Berlin Heidelberg.
- [EUC25] Agreed Cryptographic Mechanisms, Version 2.0, April 2025. European Cybersecurity Certification Group – European Union Agency for Cybersecurity (ENISA) – https://certification.enisa.europa.eu/publications/eucc-guidelines-cryptography_en.
- [Fal20] Team Falcon. Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU. <https://falcon-sign.info/>, 2020. Version 1.2 01/10/2020.
- [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure Integration of Asymmetric and Symmetric Encryption Schemes. In Michael Wiener, editor, *Advances in Cryptology – CRYPTO ’99*, pages 537–554, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
- [Fro23] Team FrodoKEM. FrodoKEM: Learning With Errors Key Encapsulation - Preliminary Standardization Proposal. <https://frodokem.org/>, 2023. Version 2023-03-14.
- [GBHLY16] Leon Groot Bruinderink, Andreas Hülsing, Tanja Lange, and Yuval Yarom. Flush, Gauss, and Reload – a Cache Attack on the BLISS Lattice-Based Signature Scheme. In *International Conference on Cryptographic Hardware and Embedded Systems*, pages 323–345. Springer, 2016.
- [GG25] François Gérard and Morgane Guerreau. High-Order and Cortex-M4 First-Order Implementations of Masked FrodoKEM. Cryptology ePrint Archive, Paper 2025/1065, 2025. <https://ia.cr/2025/1065>.
- [GMRR22] Morgane Guerreau, Ange Martinelli, Thomas Ricosset, and Mélissa Rossi. The Hidden Parallelepiped Is Back Again: Power Analysis Attacks on Falcon. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(3):141164, Jun. 2022.
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for Hard Lattices and New Cryptographic Constructions. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pages 197–206, 2008.
- [GR20] François Gérard and Mélissa Rossi. An Efficient and Provable Masked Implementation of qTESLA. In Sonia Belaïd and Tim Güneysu, editors, *Smart Card Research and Advanced Applications*, pages 74–91, Cham, 2020. Springer International Publishing.
- [GR24] Morgane Guerreau and Mélissa Rossi. A not so discrete sampler: Power analysis attacks on hawk signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(4):156178, Sep. 2024.
- [ISW03] Yuval Ishai, Amit Sahai, and David A. Wagner. Private Circuits: Securing Hardware Against Probing Attacks. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 463–481. Springer, 2003.

- [KH18] Suhri Kim and Seokhie Hong. Single Trace Analysis on Constant Time CDT Sampler and its Countermeasure. *Applied Sciences*, 8(10):1809, 2018.
- [KLS⁺23] Markus Krausz, Georg Land, Florian Stolz, Dennis Naujoks, Jan Richter-Brockmann, Tim Güneysu, and Lucie Kogelheide. To extend or not to extend: Agile Masking Instructions for PQC. Cryptology ePrint Archive, Paper 2023/1287, 2023. <https://ia.cr/2023/1287>.
- [KRR⁺18] Angshuman Karmakar, Sujoy Sinha Roy, Oscar Reparaz, Frederik Vercauteren, and Ingrid Verbauwhede. Constant-Time Discrete Gaussian Sampling. *IEEE Transactions on Computers*, 67(11):1561–1571, 2018.
- [MKK⁺23] Soundes Marzougui, Ievgan Kabin, Juliane Krämer, Thomas Aulbach, and Jean-Pierre Seifert. On the Feasibility of Single-Trace Attacks on the Gaussian Sampler using a CDT. Cryptology ePrint Archive, Paper 2023/142, 2023. <https://ia.cr/2023/142>.
- [NIS16] National Institute of Standards and Technology, Post-quantum cryptography standardization, 2016. <https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization>.
- [Pes16] Peter Pessl. Analyzing the Shuffling Side-Channel Countermeasure for Lattice-Based Signatures. In *Progress in Cryptology–INDOCRYPT 2016: 17th International Conference on Cryptology in India, Kolkata, India, December 11–14, 2016, Proceedings 17*, pages 153–170. Springer, 2016.
- [PZ03] J. Proos and C. Zalka. Shor’s discrete logarithm quantum algorithm for elliptic curves. *Quantum Inf. Comput.*, 2003:3:317–344, 2003.
- [QCZ⁺21] Yue Qin, Chi Cheng, Xiaohan Zhang, Yanbin Pan, Lei Hu, and Jintai Ding. A Systematic Approach and Analysis of Key Mismatch Attacks on Lattice-Based NIST Candidate KEMs. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, pages 92–121, Cham, 2021. Springer International Publishing.
- [Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing, STOC ’05*, page 8493, New York, NY, USA, 2005. Association for Computing Machinery.
- [RRD⁺23] Gokulnath Rajendran, Prasanna Ravi, Jan-Pieter DAnvers, Shivam Bhasin, and Anupam Chattopadhyay. Pushing the Limits of Generic Side-Channel Attacks on LWE-based KEMs - Parallel PC Oracle Attacks on Kyber KEM and Beyond. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(2):418446, Mar. 2023.
- [RRVV14] Sujoy Sinha Roy, Oscar Reparaz, Frederik Vercauteren, and Ingrid Verbauwhede. Compact and Side Channel Secure Discrete Gaussian Sampling. Cryptology ePrint Archive, Paper 2014/591, 2014. <https://ia.cr/2014/591>.
- [RSRCB20] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. Generic Side-channel attacks on CCA-secure lattice-based PKE and KEMs. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020(3):307335, Jun. 2020.

- [Saa18] Markku-Juhani O. Saarinen. Arithmetic coding and blinding countermeasures for lattice signatures - engineering a side-channel resistant post-quantum signature scheme with compact signatures. *J. Cryptogr. Eng.*, 8(1):71–84, 2018.
- [Sho94] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. *35th FOCS, IEEE Computer Society Press*, 1994:124–134, November 1994.
- [UXT⁺21] Rei Ueno, Keita Xagawa, Yutaro Tanaka, Akira Ito, Junko Takahashi, and Naofumi Homma. Curse of Re-encryption: A Generic Power/EM Analysis on Post-Quantum KEMs. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(1):296322, Nov. 2021.
- [ZLYW23] Shiduo Zhang, Xiuhan Lin, Yang Yu, and Weijia Wang. Improved Power Analysis Attacks on Falcon. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023*, pages 565–595, Cham, 2023. Springer Nature Switzerland.